Domande G2 — Sistemi Operativi (Raccolta Completa)

> *Appunti di Ricky Wanga · X @rickywanga42 · rickywanga.com*

> *52 esami analizzati (2006-2025). 17 con risposte dalle soluzioni del repo.*
>
> *Organizzate per area tematica. Le domande senza risposta sono segnate ⚠️.*
> *Risposte provenienti da: soluzioni repo cartabinaria, alibenny, PDF Notion Luizo.*

🧠 Memoria (Paginazione, Memoria Virtuale, Trashing, Rimpiazzamento)

Perché viene usata la paginazione per implementare la memoria virtuale?

> *(2022-01-17 a, 2022-02-14 a, 2022-06-01 1)*

La paginazione nasce come tecnica di allocazione della memoria, ma presenta buone qualità anche per la memoria virtuale:

- La suddivisione in pagine/frame è in linea con le necessità della memoria virtuale di fare swap-in/out dei dati non utili al momento
- La tecnica è implementabile tramite hardware semplice (molto più della segmentazione), riducendo il costo
- Riduce la frammentazione interna (pagine tipicamente piccole) ed elimina quasi completamente la frammentazione esterna
- È trasparente al programmatore, non richiede sforzo implementativo da chi scrive software (a differenza della segmentazione)
- Permette di simulare più memoria di quella fisicamente presente e usare tecniche avanzate di sicurezza

La paginazione non elimina completamente né la frammentazione interna né quella esterna ma le rende trascurabili. Perché?

> *(2014-01-22 1, 2014-02-21 1)*

La frammentazione interna è ridotta a un caso solo: l'ultima pagina di ogni processo, che in media sarà piena per metà. Con pagine piccole (es. 4KB), lo spreco è trascurabile. La frammentazione esterna è quasi eliminata perché tutti i frame hanno la stessa dimensione e qualsiasi frame libero può ospitare qualsiasi pagina — non ci sono più "buchi" inutilizzabili tra frame occupati. L'unica frammentazione esterna residua è talmente ridotta da essere irrilevante.

Quali problemi risolve la paginazione e quali la segmentazione?

> *(2014-06-03 3)*

La paginazione risolve: la frammentazione esterna (frame tutti uguali), la necessità di memoria virtuale e swap, e la complessità di allocazione dinamica. È trasparente al programmatore e supporta protezione tra processi. La segmentazione risolve: la necessità di suddividere la memoria in aree funzionalmente distinte (codice, dati, stack) con protezioni diverse, la condivisione di codice e dati tra processi, e la gestione di strutture dati a dimensione variabile. La segmentazione è visibile al programmatore e richiede attenzione esplicita.

Quali sono le differenze fra paginazione e segmentazione nella gestione della memoria?

> *(2013-01-24, 2013-02-15, 2013-06-21 a)*

Paginazione: divide lo spazio logico in blocchi di dimensione fissa (pagine) e la memoria fisica in frame della stessa dimensione. L'indirizzo è <numero-pagina, offset>. È trasparente al programmatore. Segmentazione: divide la memoria in aree di dimensione variabile con significato funzionale (text, data, stack). L'indirizzo è <nome-segmento, offset>. La segmentazione è visibile al programmatore, facilita condivisione e protezione differenziata per tipo di segmento, ma è più complessa da implementare in hardware e può causare frammentazione esterna.

Cosa sono TLB miss e page fault? Quale differenza?

> *(2018-01-22 b)*

TLB miss: la traduzione da indirizzo logico a fisico non è presente nel Translation Lookaside Buffer, la cache hardware della tabella delle pagine. Page fault: la pagina richiesta non è in memoria principale e deve essere caricata dalla memoria secondaria (swap). La differenza fondamentale è che un TLB miss non implica page fault: la pagina può essere in RAM ma semplicemente non cache-ata nel TLB. Viceversa, un page fault causa sempre TLB miss perché la pagina non è in memoria.

A cosa serve il Translation Lookaside Buffer?

> *(2023-01-18 b)*

Il TLB è una cache hardware per la tabella delle pagine, composta da registri associativi ad alta velocità. Ogni registro contiene una coppia <chiave, valore> dove la chiave è il numero di pagina logica e il valore è il frame fisico corrispondente. Il TLB riduce drasticamente il costo degli accessi in memoria: senza TLB, ogni accesso richiederebbe due accessi in RAM (uno per la tabella pagine, uno per il dato). Con TLB, la traduzione avviene in un ciclo di clock.

In un sistema a memoria virtuale un interrupt TLB-miss indica sempre page miss?

> *(2023-02-15 b)*

No, assolutamente. Il TLB-miss indica solo che la traduzione non è presente nella cache TLB in quel momento. La pagina può benissimo essere in memoria principale ma assente dal TLB perché: appena caricata, il TLB è pieno ed è stata rimossa, o appartiene a un altro processo. In questi casi basta aggiornare il TLB con la traduzione corretta senza generare page fault. Solo se la pagina non è proprio in RAM si ha un page fault.

Cosa significa per un algoritmo di rimpiazzamento essere "a stack"? Come si ripercuote sull'anomalia di Belady?

> *(2023-06-13 c)*

Dalla definizione di algoritmo a stack: aumentando il frame buffer abbiamo almeno le pagine con dimensione del frame buffer minore. Quindi **non è possibile che `m` frame causino più page fault di `n` frame** (con `n < m`). Di conseguenza, un algoritmo a stack **non può soffrire dell'anomalia di Belady**.

Perché un algoritmo di rimpiazzamento a stack non può essere soggetto ad anomalia di Belady?

> *(2022-07-20 a)*

Un algoritmo a stack non può essere soggetto ad anomalia di Belady in quanto dovrebbe avvenire page fault nel caso della memoria con più frame — ma per definizione di algoritmo a stack, aumentando i frame non si possono avere più page fault.

Dimostrare che se un algoritmo di rimpiazzamento è a stack allora non può soffrire dell'anomalia di Belady

> *(2019-06-18 b)*

Per definizione, un algoritmo a stack ha la proprietà che l'insieme M*m(t) delle pagine in memoria con m frame è sempre un sottoinsieme di M*{m+1}(t) (con m+1 frame): M*m(t) ⊆ M*{m+1}(t). Di conseguenza, quando si verifica un page fault con m+1 frame, la pagina richiesta non appartiene a M_{m+1}(t), quindi a maggior ragione non appartiene a M_m(t) — pertanto anche con m frame si ha page fault. Ma può esistere un page fault con m frame che NON si verifica con m+1 frame (perché la pagina è nel sottoinsieme aggiuntivo). Quindi: page_fault(m) ≥ page_fault(m+1). I page fault non possono aumentare all'aumentare dei frame, il che contraddice l'anomalia di Belady (che richiederebbe page_fault(m) > page_fault(m+1) per qualche m).

L'algoritmo di rimpiazzamento second chance (orologio) è a stack? Perché viene preferito a LRU?

> *(2017-05-29 c, 2021-06-23 c, 2025-01-15 c)*

Non è a stack, quindi potrebbe soffrire dell'anomalia di Belady. Viene preferito a LRU perché:

- LRU richiederebbe contatori per ogni pagina (con overflow e scansione di tutte le pagine per scegliere la vittima) oppure uno stack con 6 puntatori da aggiornare ad ogni accesso — entrambi inefficienti
- Second chance è molto più semplice da implementare: un bit di riferimento e una scansione circolare

Perché l'algoritmo LRU non viene implementato nei sistemi operativi reali?

> *(2018-09-19 a)*

Perché l'implementazione è inefficiente:

- O memorizzo contatori per ogni pagina, ma dovrei gestire l'overflow e scandire tutte le pagine in memoria per scegliere la vittima (troppi accessi in memoria)
- O uso uno stack ma ogni accesso ad una pagina dovrei modificare lo stack per mettere tale pagina in cima, aggiornando sei puntatori

Qual è la differenza tra paginatore e algoritmo di rimpiazzamento? In quali situazioni sono chiamati in causa?

> *(2019-09-13 c)*

- **Paginatore**: entra in funzione quando avviene un **page fault** (pagina richiesta non in memoria)
- **Algoritmo di rimpiazzamento**: entra in funzione quando **non ci sono frame liberi** e bisogna scegliere una pagina vittima da rimuovere. La scelta della pagina vittima è data da tale algoritmo.

Quando viene richiamato l'algoritmo del Banchiere? Cosa succede se safe / non-safe?

> *(2018-02-12 a, 2018-05-28 d, 2019-07-15 d, 2022-01-17 b, 2022-02-14 b, 2021-09-15 d)*

L'algoritmo del banchiere viene eseguito ogni volta che **non si può soddisfare immediatamente una richiesta di assegnazione**.

- **Safe**: si assegnano le risorse (è garantito che non ci sia deadlock), si aggiorna lo stato di allocazione
- **Unsafe**: c'è rischio deadlock → **non si assegnano le risorse** fino a quando non vengono rilasciate risorse sufficienti. Il processo resta in attesa.

Qual è il problema principale dell'algoritmo del banchiere che ne impedisce l'uso in un SO reale?

> *(2023-06-13 d)*

I processi **non dichiarano il massimo delle risorse** che utilizzeranno all'inizio dell'esecuzione. Si può stimare, ma non sarà mai esatto come richiede l'algoritmo.
Nota: non è corretto dire che è inefficiente — il teorema dimostra che è efficiente perché viene aggiornata solo l'ultima riga della tabella.

Quali sono le condizioni necessarie per poter applicare l'algoritmo del banchiere?

> *(2017-06-19 c)*

L'algoritmo del banchiere richiede: (1) ogni processo deve dichiarare a priori il massimo numero di risorse di ciascuna classe di cui potrà aver bisogno durante l'esecuzione; (2) il numero di processi e di classi di risorse deve essere noto e fisso; (3) il sistema deve mantenere in ogni istante lo stato di allocazione corrente (risorse assegnate a ciascun processo e risorse ancora disponibili); (4) l'algoritmo viene invocato quando una richiesta non può essere soddisfatta immediatamente, e verifica se lo stato risultante sarebbe safe.

Qual è la complessità computazionale del banchiere monovaluta e multivaluta?

> *(2023-01-18 d)*

Monovaluta (una sola classe di risorse): O(n log n) dove n è il numero di processi. Basta ordinare i processi per credito residuo n_i = c_i - p_i crescente e verificare che la sequenza sia percorribile con il saldo disponibile. Multivaluta (m classi di risorse): O(n² × m). A ogni passo si cerca tra tutti i processi non ancora soddisfatti uno le cui richieste residue siano tutte ≤ alle risorse ancora disponibili; se si trova lo si marca come soddisfatto e si rilasciano le sue risorse. Se a un passo nessun processo è soddisfacibile, lo stato è unsafe.

A cosa serve e quando viene eseguito l'algoritmo di calcolo del working set?

> *(2018-06-21 a)*

L'algoritmo di calcolo del working set serve per evitare il trashing. Sceglie quali pagine di ogni processo vanno mantenute in memoria e quali no. Viene eseguito ogni volta che avviene un page fault.

A cosa serve e quando viene eseguito un algoritmo di rimpiazzamento?

> *(2018-05-28 a)*

L'algoritmo di rimpiazzamento serve quando non ci sono frame liberi in memoria e bisogna scegliere una pagina vittima da rimuovere per far posto a una nuova pagina. Viene eseguito durante la gestione del page fault, precisamente al passo in cui il paginatore non trova un frame libero e deve liberare spazio. L'obiettivo è minimizzare il numero di page fault scegliendo una vittima che non verrà acceduta a breve.

Il calcolo del working set dipende dall'algoritmo di rimpiazzamento utilizzato? Perché?

> *(2019-07-15 b)*

No, non dipende. Il working set di un processo è definito esclusivamente dalla finestra temporale Δ e dalla sequenza di riferimenti alle pagine fatta dal processo — è una proprietà intrinseca del comportamento del processo. L'algoritmo di rimpiazzamento decide quale pagina rimuovere quando mancano frame liberi, ma il working set (l'insieme delle pagine accedute nei più recenti Δ riferimenti) è calcolato indipendentemente e serve proprio per guidare la decisione su quali pagine tenere in memoria.

Il valore della "finestra" del working set dipende dall'algoritmo di rimpiazzamento utilizzato? Perché?

> *(2019-02-14 a)*

No, non dipende. La finestra Δ è un parametro di sistema scelto in base a considerazioni architetturali (dimensione della memoria fisica, numero di processi attivi, pattern di accesso tipici), non dall'algoritmo di rimpiazzamento. Δ determina la granularità temporale con cui si osserva il comportamento del processo: un Δ troppo piccolo causa falsi negativi (pagine necessarie escluse), un Δ troppo grande causa falsi positivi (pagine inutili incluse), indipendentemente da come poi si rimpiazzano le pagine.

Perché aumentando lo spazio di swap si rischia trashing?

> *(2017-06-19 a)*

Aumentare lo spazio di swap consente di ospitare più processi in memoria virtuale, incrementando il grado di multiprogrammazione. Se la somma dei working set di tutti i processi attivi supera il numero di frame fisici disponibili, ogni processo genera continui page fault accedendo a pagine che altri processi hanno appena rimpiazzato. Il sistema entra in trashing: la CPU rimane largamente inattiva in attesa di I/O su disco, e paradossalmente lo scheduler potrebbe caricare ulteriori processi peggiorando la situazione (perché vede CPU bassa e pensa ci sia capacità residua).

Quando un sistema è in trashing il carico della CPU è alto o basso, perché?

> *(2019-05-18 a)*

Il carico della CPU è basso. In trashing, il sistema spende più tempo nella paginazione (I/O su disco) che nell'esecuzione effettiva dei processi. I processi sono quasi sempre bloccati in attesa che i page fault vengano serviti, mentre la CPU resta inattiva in attesa del completamento dell'I/O. Questa è la trappola del trashing: lo

scheduler, rilevando CPU bassa, può decidere di caricare altri processi, aumentando ulteriormente la contesa per i frame e peggiorando il trashing.

Per risolvere un problema di trashing è necessario terminare forzatamente dei processi?

> *(2020-01-15 b, 2020-02-20 b)*

No, non è necessario uccidere processi. Per uscire dal trashing (che si verifica quando molti processi generano innumerevoli page fault accedendo a indirizzi sparsi tra memoria centrale e secondaria) è sufficiente sospendere alcuni processi, in attesa che altri completino e liberino aree di memoria. Riducendo i processi attivi si riducono i cache miss. Lo svantaggio è la riduzione del parallelismo.

Quali contromisure quando la somma dei working set supera il numero di frame?

> *(2018-02-12 b)*

Si deve ridurre il grado di multiprogrammazione: sospendere uno o più processi (spostandoli completamente in swap area) finché altri processi non completano e liberano i loro frame. In alternativa si può ridurre la finestra Δ per restringere i working set, ma questo rischia di escludere pagine realmente necessarie (falsi negativi), causando ulteriori page fault. La sospensione di processi è la contromisura più sicura, anche se riduce temporaneamente il parallelismo.

Quale supporto hardware serve per implementare memoria virtuale tramite paginazione a richiesta? Perché?

> *(2014-07-16 a)*

Serve una MMU (Memory Management Unit) per tradurre indirizzi logici in fisici usando la tabella delle pagine. Serve un TLB per velocizzare la traduzione senza raddoppiare gli accessi in memoria. Servono bit di stato nelle entry della tabella pagine: valid/invalid bit (per distinguere pagine in memoria da pagine in swap), dirty bit (per sapere se la pagina è stata modificata e va riscritta in swap), reference bit (per gli algoritmi di rimpiazzamento). Serve infine un meccanismo di interrupt per segnalare i page fault alla CPU quando si accede a una pagina invalida.

Se un sistema ha molta RAM può non servire la memoria virtuale. Ha senso usare comunque la paginazione?

> *(2015-01-20 a)*

Sì, ha senso. La paginazione offre vantaggi che vanno oltre la memoria virtuale: elimina la frammentazione esterna (tutti i frame sono uguali e intercambiabili), semplifica l'allocazione della memoria fisica, fornisce protezione tra processi (ogni processo ha la propria tabella pagine con permessi di accesso), e permette la condivisione efficiente di pagine tra processi (es. codice di librerie condivise mappato una sola volta in RAM ma visibile a più processi). Anche senza swap, la paginazione migliora la gestione della memoria.

In quali casi è bene NON usare la memoria virtuale?

> *(2018-07-17 a)*

È bene non usare la memoria virtuale quando:

- La memoria secondaria ha dimensioni analoghe a quelle della memoria primaria
- Non abbiamo bisogno di più memoria di quella fisica principale
- Viene a cadere il principio di località spaziale/temporale
- Si punta ad un sistema concettualmente minimale

Quali problemi può porre l'implementazione di un SO con memoria virtuale e controller DMA?

> *(2023-01-18 c)*

Il DMA accede alla memoria usando indirizzi fisici, mentre il SO con memoria virtuale lavora con indirizzi logici. Durante un trasferimento DMA, le pagine del processo coinvolto potrebbero essere rimpiazzate o spostate dal

paginatore (es. a causa di page fault di altri processi), causando corruzione dei dati. La soluzione è il page pinning: le pagine coinvolte in un trasferimento DMA vengono bloccate in memoria fisica finché il DMA non ha completato. Questo richiede cooperazione tra sottosistema di memoria virtuale e driver DMA.

Quali sono i casi di frammentazione interna ed esterna con la paginazione?

> *(2015-02-14 1)*

Frammentazione interna: si verifica esclusivamente nell'ultima pagina di ogni processo, che in media è riempita per metà. Poiché le pagine sono piccole (tipicamente 4KB), lo spreco per processo è trascurabile. Frammentazione esterna: praticamente inesistente, perché tutti i frame hanno la stessa dimensione e qualsiasi frame libero può ospitare qualsiasi pagina. Non si formano mai "buchi" inutilizzabili tra frame occupati, risolvendo il problema principale dell'allocazione a partizioni dinamiche.

Applicando paginazione + segmentazione aumenta la frammentazione interna?

> *(2017-07-17 b)*

Sì, aumenta. Con la sola segmentazione, ogni segmento ha esattamente la dimensione richiesta dal processo — non c'è frammentazione interna. Combinando segmentazione e paginazione, ogni segmento viene suddiviso in pagine di dimensione fissa: l'ultima pagina di ogni segmento sarà parzialmente riempita, introducendo frammentazione interna per ciascun segmento. Invece di una sola pagina parzialmente vuota per processo (come nella paginazione pura), se ne hanno tante quanti sono i segmenti.

Come funziona la compattazione di memoria? Quali sono i difetti?

> *(2025-02-11 d)*

La compattazione sposta fisicamente tutti i processi in memoria principale per riunire le aree libere frammentate in un unico blocco contiguo. Richiede che il codice sia rilocabile dinamicamente (binding a esecuzione) e hardware di rilocazione nella MMU. Difetti: è estremamente onerosa in termini di tempo (spostamento di grandi quantità di dati), blocca l'intero sistema durante l'operazione, consuma cicli di CPU senza produrre lavoro utile, e negli scenari peggiori va eseguita frequentemente se i processi variano dimensione.

Perché è meglio la paginazione della compattazione di memoria?

> *(2022-06-21 c)*

La paginazione elimina del tutto la necessità di compattazione. Non essendoci frammentazione esterna significativa (ogni frame libero può ospitare qualsiasi pagina), non serve mai spostare fisicamente le pagine per "fare spazio". La paginazione gestisce l'allocazione in modo continuo e trasparente con overhead costante e prevedibile. La compattazione invece è un'operazione episodica ma estremamente costosa, che blocca il sistema e richiede hardware di rilocazione. La paginazione è quindi più scalabile ed efficiente.

📂 File System (i-node, FAT, Allocazione, Link, Journaling, RAID)

Come si calcola la lunghezza massima di un file in un file system di tipo UNIX (ext2)?

> *(2018-05-28 b, 2022-06-21 a, 2022-09-06 b)*

Si calcola sommando i contributi dei puntatori nell'i-node. Con dimensione blocco B e puntatori da P byte: i K puntatori diretti (tipicamente 12) coprono K × B byte. Il puntatore indiretto singolo indirizza B/P blocchi → (B/P) × B. L'indiretto doppio: $(B/P)^2 \times B$. L'indiretto triplo: $(B/P)^3 \times B$. Esempio con B=1KB e P=4B: 12KB + 256KB + 64MB + 16GB ≈ 16GB. Con blocchi da 4KB e puntatori da 4B, la dimensione massima è molto maggiore (nell'ordine dei TB).

Come si calcola la lunghezza massima di un file in FAT?

> *(2018-06-21 b)*

Dipende dal numero massimo di cluster indirizzabili e dalla dimensione del cluster. Con FAT32: la FAT può indirizzare circa 2^{28} cluster (alcuni valori riservati). La dimensione massima del file è quindi: numero_massimo_cluster × dimensione_cluster. Con cluster da 4KB: ~1TB. Con cluster da 32KB: ~8TB. Il limite pratico per FAT32 è 4GB per file (limite storico a 32 bit per la dimensione nei metadati), anche se esistono estensioni. FAT è intrinsecamente limitato dalla dimensione fissa della tabella e dalla granularità dei cluster.

Perché il nome del file NON è memorizzato nell'i-node nei file system UNIX (ext2/3/4)?

> *(2021-07-21 a)*

Perché tramite hard link potrei avere due (o più) nomi diversi che puntano allo stesso i-node. Il nome del file è memorizzato nelle directory, non nell'i-node.

Cosa succede se il link count di un i-node è errato (maggiore o minore del reale)?

> *(2017-05-29 a, 2025-01-15 a)*

Se maggiore del reale: il file system pensa che il file abbia più riferimenti di quanti ne ha realmente. Anche dopo la rimozione di tutti i link effettivi, il file non verrà mai deallocato perché il contatore non raggiunge zero. Lo spazio su disco è perso permanentemente (memory leak del file system). Se minore del reale: il file system potrebbe deallocare prematuramente l'i-node e i blocchi dati quando il contatore raggiunge zero, mentre esistono ancora directory entry (hard link) validi che puntano a quell'i-node. Questi diventano dangling reference: puntano a blocchi ormai liberi o riassegnati, causando potenziale corruzione dati.

Qual è il numero di link indicati nell'i-node di una directory? Perché?

> *(2019-02-14 c)*

Il link count di una directory è almeno 2: il link dalla directory padre (la entry ".." nella directory stessa punta al padre, e il padre ha una entry che punta a questa directory) e il link da sé stessa (la entry "."). Inoltre, ogni sottodirectory aggiunge 1 al link count (perché la entry ".." in ogni sottodirectory punta a questa directory). Quindi: link_count = 2 + numero_di_sottodirectory. Questo spiega perché il comando `ls -l` mostra tipicamente un link count >= 2 per le directory.

Come vengono memorizzati i link fisici (hard link) in un filesystem?

> *(2019-05-18 d)*

Un hard link è implementato come una normale directory entry in cui il nome del link viene associato allo stesso numero di i-node del file originale. Il contatore di riferimenti (link count) nell'i-node viene incrementato di 1. Non esiste alcuna distinzione tra il link "originale" e l'hard link: entrambi sono directory entry equivalenti che puntano allo stesso i-node e quindi agli stessi blocchi dati. La cancellazione di uno dei due decrementa il link count; solo quando raggiunge zero l'i-node e i blocchi vengono deallocati.

Come vengono memorizzati i link simbolici in un filesystem?

> *(2019-06-18 d)*

Un link simbolico è un file speciale con un proprio i-node di tipo symbolic link (l). Il contenuto del link simbolico è il percorso (assoluto o relativo) del file target, memorizzato come stringa. Se il percorso è sufficientemente corto (tipicamente < 60 byte), viene memorizzato direttamente nell'i-node stesso senza allocare blocchi dati. Se più lungo, occupa uno o più blocchi dati come un file normale. Quando il SO risolve un link simbolico, legge il percorso dall'i-node e lo segue per trovare il file target.

Vantaggi della FAT rispetto all'allocazione concatenata?

> *(2017-06-19 d, 2022-06-01 3)*

Il file system FAT mantiene i puntatori ai blocchi in una tabella centralizzata (File Allocation Table). Vantaggi:

- Per una `lseek`, il disco meccanico deve fare molti meno spostamenti della testina (legge la tabella invece di saltare da un blocco all'altro)

- Lascia tutta la dimensione del blocco a disposizione per i dati (nessun spazio perso per puntatori nei blocchi), diminuendo la frammentazione interna

Perché il file system FAT è più efficiente dell'allocazione concatenata coi puntatori nei blocchi?

> *(2022-06-01 3)*

Con l'allocazione concatenata, ogni blocco dati contiene un puntatore al blocco successivo, riducendo lo spazio utile per i dati e costringendo la testina del disco a seguire fisicamente la catena blocco per blocco. FAT risolve entrambi i problemi: la tabella centralizzata (spesso cache-ata in RAM) permette di seguire la catena logicamente senza spostamenti fisici della testina, e i blocchi dati sono interamente disponibili per i dati (nessun byte sprecato per puntatori). Inoltre, con la FAT in cache, l'accesso diretto a qualsiasi posizione del file è immediato.

Perché il file system FAT viene ancora utilizzato?

> *(2018-07-17 b)*

- **Implementazione semplice**
- **Interoperabilità/portabilità** (vecchi sistemi Windows, chiavette USB, schede SD)
  I restanti vantaggi sono superati da file system ottimizzati (ext2/3/4) e non costituiscono motivi validi per usare ancora FAT in nuovi sistemi.

È appropriato usare FAT per file soggetti a frequenti aggiornamenti e variazione di dimensione?

> *(2019-07-15 c)*

Non è la scelta migliore. FAT tende a causare significativa frammentazione esterna quando i file crescono e decrescono frequentemente: i nuovi cluster vengono allocati dove capita (primo libero), creando catene sparse sul disco che degradano le performance di accesso sequenziale. Inoltre FAT non ha journaling: un crash durante un aggiornamento può lasciare la FAT e i dati in stato incoerente, richiedendo una lunga esecuzione di fsck/scandisk per il recupero. File system con journaling (ext3/4, NTFS) o copy-on-write (ZFS, Btrfs) sono molto più adatti.

Fornire esempi di file system con allocazione contigua. Perché sarebbe inefficiente usare altri metodi?

> *(2018-01-22 c, 2022-01-17 c, 2022-02-14 c)*

Esempi: ISO 9660 per CD/DVD-ROM e file system per nastri magnetici. Su questi supporti, l'allocazione contigua è ottimale perché: sono supporti a sola lettura o ad accesso strettamente sequenziale, quindi la possibilità di crescita del file è irrilevante o assente. L'allocazione contigua elimina la necessità di strutture dati ausiliarie (FAT, blocchi indice, i-node) che su un CD-ROM o nastro sarebbero solo overhead senza offrire benefici. Altri metodi complicherebbero inutilmente l'implementazione.

Confrontare allocazione concatenata e allocazione indicizzata nei file system

> *(2013-01-24, 2013-02-15, 2013-06-21 c, 2023-02-15 d)*

Allocazione concatenata: ogni blocco contiene un puntatore al successivo. Vantaggi: nessuna frammentazione esterna, file può crescere indefinitamente, semplice. Svantaggi: accesso diretto lentissimo (scorrere tutta la catena), spazio nel blocco perso per il puntatore, vulnerabilità (un blocco corrotto rompe la catena). Allocazione indicizzata: un blocco indice contiene tutti i puntatori ai blocchi dati. Vantaggi: accesso diretto veloce (indice caricato in memoria), nessuno spazio perso nei blocchi dati. Svantaggi: dimensione massima del file limitata dall'indice, overhead per file piccoli (un intero blocco indice anche per pochi byte).

Cosa sono linking dinamico e loading dinamico?

> *(2012-06-15)*

Loading dinamico: le routine di libreria risiedono su disco in formato rilocabile e vengono caricate in memoria solo quando vengono effettivamente richiamate durante l'esecuzione. Linking dinamico: il collegamento (linking) delle routine di libreria al programma viene posticipato al primo riferimento durante l'esecuzione, anziché essere risolto

in fase di compilazione. Una singola istanza della libreria condivisa in memoria può servire tutti i processi. Il linking dinamico permette anche il caricamento di plugin a runtime tramite dlopen.

Vantaggi e svantaggi delle librerie dinamiche (DLL)?

> *(2019-09-13 d)*

Vantaggi:

- Il programma occupa meno spazio in memoria (la libreria è condivisa tra processi)
- Aggiornamenti retroattivi: basta aggiornare la libreria e tutti i processi che la usano avranno le funzioni aggiornate
  Svantaggi:
- Si può aggiornare una libreria solo se è retrocompatibile
- Programmi diversi potrebbero richiedere versioni diverse (DLL hell)

Quali sono i benefici del linking dinamico?

> *(2017-09-11 d)*

Il linking dinamico offre tre benefici principali. Primo, l'eseguibile su disco è più piccolo perché non include il codice delle librerie. Secondo, in memoria una sola copia della libreria è condivisa tra tutti i processi che la usano, risparmiando RAM. Terzo, gli aggiornamenti delle librerie sono retroattivi: basta installare la nuova versione del file .so e tutti i programmi che la caricano dinamicamente useranno automaticamente la versione aggiornata, senza necessità di ricompilare i programmi stessi.

Il journaling protegge i dati in caso di guasto?

> *(2018-09-19 b)*

Il journaling permette di ripristinare il filesystem ad uno stato coerente, ma:

- Le transazioni in coda (non confermate) non vengono recuperate e vengono perse
- Il journaling non protegge da guasti meccanici del disco
- La ricostruzione della coerenza tramite journaling risolve solo alcuni tipi di incoerenze

Anche con journaling si possono perdere informazioni? In quali casi?

> *(2018-09-19 b, 2017-07-17 a)*

Sì. Le transazioni che erano in coda (non ancora confermate) al momento del guasto vengono perse. Inoltre il journaling non protegge da guasti meccanici del disco.

RAID: conviene RAID1 o RAID5 con lo stesso numero di dischi? Differenze di sicurezza?

> *(2020-01-15 c, 2020-02-20 c)*

A parità di dischi, RAID5 consente di memorizzare più dati (resa ~N-1 dischi) rispetto a RAID1 (resa 50% per la duplicazione). Entrambi tollerano il guasto di 1 disco. RAID1 usa duplicazione, RAID5 usa bit di parità distribuiti. RAID1 è un caso particolare di RAID4 con due dischi. Con RAID1 è statisticamente più probabile che la rottura di due dischi non causi perdita dati (perché i dati sono duplicati su tutto il pool).

Dati 4 dischi da 1GiB, quanto spazio in RAID5 e quanto in RAID10?

> *(2023-06-13 b)*

- RAID5: 3 dischi dati + 1 parità = 3GiB utilizzabili
- RAID10: 2 dischi in striping × 2 (mirroring) = 2GiB utilizzabili
  RAID5 ha più spazio disponibile di RAID10.

Ha senso RAID5 con solo 2 dischi?

> *(2019-09-13 b)*

Non avrebbe molto senso. A prescindere da come sono distribuiti gli strip di parità, avremmo l'equivalente di un disco dati e uno di parità. Quello di parità sarebbe semplicemente la copia dell'altro disco (non ci sono altri dischi con cui calcolare tale parità). Sarebbe equivalente a RAID1.

RAID1 e RAID5 tollerano entrambi il guasto di un solo disco. Quando RAID1 e quando RAID5?

> *(2018-02-12 c)*

RAID1 (mirroring) è preferibile quando: il numero di dischi è ridotto (2-4), le performance di lettura sono critiche (letture parallele da entrambe le copie), la semplicità implementativa è importante, o il carico di scrittura è molto elevato (RAID1 non ha overhead di calcolo parità). RAID5 è preferibile quando: lo spazio di archiviazione è prioritario (resa N-1), si hanno almeno 3 dischi, il carico è prevalentemente di lettura, e il budget per i dischi è limitato. RAID5 ha overhead di calcolo XOR sulle scritture che può diventare un collo di bottiglia.

RAID0 non tollera guasti. Quali sono i casi di utilizzo?

> *(2024-06-25 c)*

RAID0 (striping puro senza ridondanza) si usa quando le performance di I/O sono l'unica priorità e i dati non sono critici o sono facilmente ripristinabili. Casi tipici: cache di sistema e file temporanei, scratch space per rendering video o editing audio dove i dati intermedi non vanno preservati, database di sviluppo e testing, applicazioni scientifiche dove i dati sono ricostruibili. Spesso RAID0 viene combinato con RAID1 in configurazione RAID10 per ottenere sia performance che ridondanza.

A cosa serve il RAID? È necessario fare backup dei dati memorizzati in RAID?

> *(2017-09-11 c, 2018-07-17 c)*

I sistemi RAID producono ridondanza che permette il recupero di dati danneggiati/persi in caso di guasto di uno o più dischi (a seconda del livello). Tuttavia, potrebbe essere comunque necessario fare backup esterno, poiché c'è sempre la possibilità di un fallimento generalizzato a tutta la batteria di dischi. Inoltre RAID non protegge da errori software o cancellazioni accidentali.

Perché quando si usano SSD i file system vengono configurati con noatime?

> *(2022-07-20 d)*

Con gli SSD si cerca sempre di ottimizzare i cicli di scrittura (che non sono infiniti). La scrittura dell'informazione di ultimo accesso aumenterebbe il numero di cicli consumati, diminuendo la durata del disco. Con gli SSD il tempo di ultimo accesso non è utile quanto con gli HDD.

Quali operazioni compie un file system basato su i-node quando l'utente digita `ln /etc/passwd /tmp`?

> *(2014-06-16 b)*

Il SO risolve il percorso `/etc/passwd` attraversando le directory per trovare l'i-node del file sorgente. Verifica che l'utente abbia permessi di scrittura sulla directory `/tmp`. Crea una nuova directory entry in `/tmp` con il nome `passwd` che punta allo stesso numero di i-node del file originale. Incrementa di 1 il contatore di riferimenti (link count) nell'i-node di `/etc/passwd`. Se è un hard link, dopo l'operazione entrambi i percorsi sono equivalenti e indistinguibili: condividono lo stesso i-node e gli stessi blocchi dati.

A cosa serve partizionare un disco? Esempi pratici.

> *(2018-01-22 a)*

Serve a suddividere logicamente lo spazio disco per organizzazione, sicurezza e performance. Esempi pratici: separare il sistema operativo dai dati utente (così reinstallando l'SO non si perdono i documenti), isolare partizioni con policy diverse (es. `/home` con quote disco, `/tmp` con noexec per sicurezza), supportare multi-boot (più SO sullo stesso disco), dedicare una partizione separata allo swap per performance, confinare i danni in caso di corruzione del file system (un fsck su `/var` non blocca il sistema su `/`).

🔒 Sicurezza (ACL, Capability, Password, Buffer Overflow, Virus/Worm)

Perché revocare una capability è più difficile che revocare un'ACL?

> *(2017-05-29 d, 2020-02-20 d, 2022-09-06 c, 2025-01-15 d)*

La capability è memorizzata dal processo stesso, non in un'area di memoria del sistema. Di conseguenza è più difficile revocare l'autorizzazione perché il sistema non ha un punto centralizzato dove rimuoverla. Con le ACL, invece, i diritti sono memorizzati in strutture dati del sistema (es. nell'i-node del file), quindi revocare significa semplicemente modificare quella struttura.

Caratteristiche negative dell'uso delle ACL per la memorizzazione delle autorizzazioni

> *(2021-06-23 d)*

Le ACL sono memorizzate nell'oggetto (es. nell'i-node del file), quindi a ogni accesso il sistema deve consultare la lista per verificare i permessi — la verifica richiede scansione. Con molti utenti e gruppi, le ACL possono crescere notevolmente consumando spazio su disco e tempo di verifica. La gestione è centralizzata nell'oggetto, il che va bene per la revoca ma rende complesso determinare "cosa può fare un utente" (bisogna scandire tutti gli oggetti). Le ACL non si adattano bene a sistemi distribuiti dove l'oggetto non è localmente accessibile.

Nella memorizzazione delle capability serve crittografia a singola chiave o a doppia chiave? Perché?

> *(2014-01-22 4, 2014-02-21 4)*

Serve crittografia a singola chiave (simmetrica). La capability è emessa e verificata dalla stessa entità (il kernel): il kernel cifra le informazioni <oggetto, diritti> con la propria chiave segreta e consegna il "biglietto" al processo. Quando il processo presenta la capability per accedere all'oggetto, il kernel la decifra con la stessa chiave e verifica che sia autentica e non contraffatta. La crittografia asimmetrica sarebbe superflua (non c'è scambio di chiavi tra entità diverse) e molto più costosa computazionalmente.

Per implementare un servizio di autorizzazione di tipo capability è meglio crittografia simmetrica o a chiave pubblica?

> *(2022-06-21 d)*

Meglio crittografia simmetrica. Il kernel detiene la chiave segreta, cifra la capability (oggetto + diritti) e la consegna al processo. All'atto dell'accesso, il kernel decifra la capability con la stessa chiave per verificarne l'autenticità. La crittografia a chiave pubblica non offre vantaggi in questo scenario: il kernel è sia emittente che verificatore, non c'è necessità di distribuire chiavi pubbliche, e l'asimmetrica ha costi computazionali molto superiori che introdurrebbero latenza inappropriata a ogni controllo di accesso.

Quale tipo di crittografia è necessaria per implementare capability? Perché?

> *(2021-05-26 c)*

È necessaria crittografia a singola chiave (simmetrica). Il motivo è che la capability deve essere emessa e successivamente verificata dal kernel: il kernel cifra la tupla <oggetto, diritti di accesso> con una chiave segreta nota solo a lui e la consegna al processo. Se la capability non fosse cifrata, il processo potrebbe falsificarla (modificando oggetto o diritti) per ottenere accessi non autorizzati. Con la cifratura simmetrica, il kernel può sempre decifrare e verificare l'integrità della capability quando il processo la presenta.

A cosa serve il meccanismo del "sale" (salt) nelle password? Perché /etc/shadow invece di /etc/passwd?

> *(2017-07-17 d, 2018-05-28 c, 2022-06-01 4)*

Salt: si appende una quantità di byte random alla password prima di passarla alla one-way function che genera l'hash. L'hash va memorizzato assieme al suo sale. Questo previene attacchi a dizionario: un dizionario precalcolato di hash è inutile perché ogni password ha un sale diverso. L'attaccante è costretto a calcolare gli hash sul momento, neutralizzando l'attacco.

Perché `/etc/shadow` e non `/etc/passwd`: per evitare che durante eventuali attacchi le password (hashate) siano accessibili e possano essere rubate. `/etc/passwd` è leggibile da tutti gli utenti, mentre `/etc/shadow` è accessibile solo agli amministratori di sistema.

Come funziona un attacco buffer overflow? Come è stato possibile eseguire codice macchina?

> *(2021-07-21 b)*

Sovrascrivendo l'indirizzo di ritorno sullo stack, l'attaccante può far ritornare il programma a un indirizzo dello stack che contiene codice arbitrario scritto dall'attaccante stesso. È stato possibile storicamente perché non c'erano misure di sicurezza come:

- Stack non eseguibile (NX bit)
- Canarino (stack canary): valore sentinella che se sovrascritto fa terminare il programma con errore

Perché è bene che l'area dello stack non sia eseguibile?

> *(2024-06-25 b)*

Rendere lo stack non eseguibile (NX bit / DEP) previene una delle tecniche più comuni di sfruttamento dei buffer overflow: l'iniezione di shellcode. Se l'attaccante sovrascrive il buffer e l'indirizzo di ritorno per saltare a codice malevolo iniettato sullo stack, la CPU solleverà un'eccezione perché tenta di eseguire codice in un'area di memoria marcata come dati. Questo non blocca tutti gli attacchi (esistono tecniche come return-to-libc e ROP), ma è una difesa fondamentale di primo livello che alza notevolmente la barriera per l'attaccante.

Differenze fra virus e worm? Come ci si difende?

> *(2018-06-21 c)*

Il virus è un frammento di codice che si attacca a programmi esistenti modificandoli; si diffonde quando un utente esegue il programma infetto e richiede interazione umana. Il worm è un programma autonomo che si replica attraverso la rete sfruttando vulnerabilità di sistema, senza alcuna interazione umana. Difese: per entrambi, mantenere SO e software aggiornati con patch di sicurezza, usare antivirus/antimalware con firme aggiornate, applicare il principio del minimo privilegio. Per i worm in particolare: firewall per bloccare porte non necessarie, segmentazione di rete, disabilitazione di servizi non essenziali.

Metodologie per minimizzare i rischi di attacco tramite worm?

> *(2023-02-15 c)*

Applicare tempestivamente le patch di sicurezza a SO e servizi di rete esposti. Configurare firewall restrittivi (default deny) limitando le porte in ascolto allo stretto necessario. Disabilitare servizi di rete non utilizzati. Applicare il principio del minimo privilegio: i processi di rete non devono girare come root. Segmentare la rete (VLAN, DMZ) per limitare la diffusione laterale in caso di compromissione. Usare sistemi di intrusion detection/prevention (IDS/IPS) per rilevare comportamenti anomali. Mantenere backup regolari e offline per il recovery.

💾 I/O, DMA, Interrupt, Spinlock

Perché DMA viene usato per dischi e non per terminali?

> *(2021-06-23 a)*

Il DMA viene utilizzato per dischi e memoria secondaria perché questi dispositivi trasferiscono interi blocchi di dati (tipicamente KB o MB) in una singola operazione. Il setup del DMA (programmazione del controller con indirizzo, dimensione, direzione) ha un costo fisso che viene ammortizzato su grandi trasferimenti. Per i terminali, i trasferimenti sono tipicamente di pochi byte o caratteri alla volta: il costo di programmazione del controller DMA supererebbe il costo del trasferimento stesso, rendendo il DMA controproducente. Per dispositivi a basso volume di dati, l'interrupt-driven I/O o il semplice polling sono più efficienti.

Differenze fra device driver con DMA e senza DMA?

> *(2014-06-03 1, 2019-06-18 c)*

Senza DMA, il device driver usa Programmed I/O o Interrupt-driven I/O: la CPU è direttamente coinvolta nel trasferimento di ogni byte tra dispositivo e memoria. Con DMA, il driver si limita a programmare il controller DMA (specificando indirizzo sorgente, destinazione, dimensione e direzione) e poi torna a eseguire altro. Il controller DMA trasferisce autonomamente l'intero blocco dati direttamente tra dispositivo e memoria centrale, e al termine genera un interrupt. Il driver con DMA è più complesso da configurare ma libera la CPU per tutto il trasferimento, migliorando drasticamente le performance per I/O di grandi volumi.

È necessario il supporto DMA per implementare un device driver interrupt driven?

> *(2014-09-24 a, 2017-09-11 b)*

No, non è necessario. Un driver interrupt-driven può funzionare perfettamente senza DMA. Il flusso tipico è: la CPU carica i parametri nei registri del controller del dispositivo, il dispositivo esegue l'operazione mentre la CPU passa ad altro, e al completamento il dispositivo genera un interrupt. Nell'interrupt handler, la CPU copia i dati dal buffer del dispositivo alla memoria (o viceversa) tramite Programmed I/O, un byte/parola alla volta. Questo approccio è adatto a dispositivi con bassi volumi di dati (terminali, mouse, tastiere). Il DMA è un'ottimizzazione per dispositivi ad alto throughput, non un requisito per il modello interrupt-driven.

Può esistere un controller DMA che non genera interrupt? Come sarebbe il device driver?

> *(2025-02-11 b)*

Sì, può esistere. Il device driver dovrebbe usare polling: dopo aver programmato il controller DMA con i parametri del trasferimento, invece di aspettare un interrupt, periodicamente legge un registro di stato del controller per determinare se il trasferimento è terminato. Il driver potrebbe fare busy waiting (loop continuo sul registro di stato) o polling a intervalli regolari (controllo periodico). Questa soluzione è inefficiente perché vanifica il principale vantaggio del DMA (lasciare la CPU libera) ed è giustificabile solo in sistemi embedded minimali senza controller di interrupt o dove la latenza dell'interrupt è inaccettabile.

Quali operazioni compie un device driver per iniziare una I/O? Come viene rilevato il completamento?

> *(2023-02-15 a)*

Per iniziare una I/O, il driver: (1) riceve dal kernel una richiesta generica (es. "leggi N byte dal disco all'indirizzo X"), (2) la traduce in comandi specifici del dispositivo, (3) scrive i comandi e i parametri (indirizzo del buffer, dimensione dati) nei registri di controllo del controller, (4) se usa DMA, programma anche il controller DMA. Il completamento viene rilevato tramite un interrupt hardware generato dal controller del dispositivo: l'interrupt handler del driver legge il registro di stato del controller per verificare la presenza di errori, e se tutto è andato bene segnala al kernel il completamento dell'operazione.

Perché gli interrupt hanno reso i SO più efficienti?

> *(2019-09-13 a, 2022-01-17 d, 2022-02-14 d)*

Tramite la tecnologia degli interrupt il processore **evita di fare busy waiting** (attesa attiva) per il completamento delle operazioni di I/O. Il processore può eseguire un altro processo mettendo in pausa quello che aveva fatto richieste I/O, tornando a eseguirlo quando l'interrupt di completamento viene lanciato.

Perché è necessario usare spinlock in sistemi multiprocessore per kernel SMP?

> *(2017-05-29 b, 2024-06-25 d, 2025-01-15 b)*

In un sistema multiprocessore SMP, più CPU possono eseguire codice kernel simultaneamente e accedere alle stesse strutture dati condivise (code dei processi, tabelle delle pagine, buffer cache, etc.). Uno spinlock protegge queste sezioni critiche: una CPU che tenta di acquisire un lock già occupato rimane in un loop di busy waiting finché il lock non viene rilasciato. In kernel mode non si può mettere un thread in sleep (non c'è un PCB di processo utente da sospendere), quindi l'attesa attiva è l'unica opzione praticabile per garantire mutua esclusione nel kernel. L'alternativa (disabilitare gli interrupt) blocca tutte le CPU.

Quali sono gli interrupt hardware più frequenti e quali le trap più frequenti?

> *(2023-01-18 a)*

Interrupt hardware più frequenti: (1) interrupt del timer/clock — usato per lo scheduling preemptive, time slice e aggiornamento dell'orologio di sistema, generato a frequenza fissa (es. 100-1000 Hz); (2) interrupt del controller disco — segnala il completamento di operazioni I/O su memoria secondaria; (3) interrupt di rete — segnala l'arrivo di pacchetti sulla scheda di rete. Trap più frequenti: (1) page fault — accesso a pagina non presente in memoria principale; (2) system call — richiesta di servizi al kernel da parte di processi utente; (3) eccezioni aritmetiche — divisione per zero, overflow.

Cosa succede quando un processo utente tenta un'operazione illegale (es. divisione per zero)? Cosa prevede POSIX?

> *(2021-06-23 b)*

Viene generata una trap. La trap viene intercettata dal SO che sceglie come gestirla: solitamente il processo viene ucciso (con segnale SIGFPE). Il processo potrebbe essere equipaggiato di un exception handler; in tal caso l'esecuzione potrebbe essere trasferita ad esso (l'handler sarebbe salvato nel PCB).

⏱️ Scheduling (Round Robin, Priorità, Batch vs Interattivo)

Uno scheduler round-robin può essere inadatto per applicazioni multimediali (molto I/O)?

> *(2020-01-15 a, 2020-02-20 a)*

Uno scheduler round-robin inserisce un processo in attesa di I/O in fondo alla coda ready una volta sbloccato. Un processo con alto numero di richieste I/O (come un'applicazione multimediale) potrebbe trovarsi troppo tempo in coda ad attendere, specialmente in un sistema affollato. Una soluzione è inserire le applicazioni con alto I/O in una coda di priorità più alta.

È sensato usare Round Robin per macchine batch? Perché?

> *(2022-06-01 2)*

No. Lo scheduler Round Robin è pensato per programmi altamente interattivi (non si vuole far aspettare l'utente). I sistemi batch eseguono lavori in differita, senza interazione umana. In questo scenario i continui context-switch causerebbero solo un degrado delle performance senza alcun beneficio.

Dimezzare il time slice del round-robin per servire meglio i processi interattivi: quali effetti collaterali?

> *(2021-07-21 d)*

Se il time slice è troppo breve, l'alto numero di context switch potrebbe pesare troppo sull'efficienza del sistema (overhead di commutazione).

Cosa provoca la scelta errata del quanto di tempo in uno scheduler round-robin?

> *(2017-06-19 b)*

Quanto troppo breve: eccessivo overhead di context switch. La CPU passa una frazione significativa del tempo a salvare e ripristinare lo stato dei processi (registri, MMU, TLB) anziché eseguire codice utile. Il rapporto tra tempo di esecuzione e tempo di commutazione diventa inefficiente. Quanto troppo lungo: il sistema perde reattività. Un processo può monopolizzare la CPU per tutto il quanto, facendo attendere a lungo i processi interattivi. Se il quanto tende a infinito, Round Robin degenera in FCFS/FIFO, vanificando l'equità e la reattività che lo caratterizzano.

Lo scheduler sceglie fra i processi ready. Cosa succede quando la ready queue è vuota?

> *(2018-06-21 d, 2021-09-15 a)*

Quando la coda ready è vuota (tutti i processi sono in attesa di I/O o terminati), lo scheduler esegue l'idle task (o processo nullo). L'idle task è un loop minimale che tiene la CPU occupata senza consumare risorse significative, tipicamente eseguendo solo l'istruzione HLT per mettere la CPU in stato di basso consumo. Appena un interrupt (es. completamento I/O, timer) rende pronto un processo, lo scheduler viene invocato e riprende l'esecuzione normale commutando al processo ready.

Confrontare algoritmi di scheduling SJN/SJF e Round Robin

> *(2014-06-16 a)*

SJF (Shortest Job First) esegue per primo il processo con CPU burst più breve. Vantaggi: minimizza il tempo medio di attesa (è ottimale sotto questo criterio), throughput elevato. Svantaggi: richiede stima dei burst futuri (approssimazione con media esponenziale), può causare starvation per processi con burst lunghi, non è adatto a sistemi interattivi. Round Robin assegna un quanto di tempo uguale a ogni processo in ordine circolare. Vantaggi: equità garantita (nessuna starvation), buona reattività per sistemi interattivi. Svantaggi: tempi medi di attesa più alti di SJF, overhead di context switch, performance sensibili alla scelta del quanto.

Differenza fra STF (Shortest Time First) e SRTF (Shortest Remaining Time First)?

> *(2017-07-17 c)*

STF (o SJF non-preemptive): una volta che un processo inizia l'esecuzione del suo CPU burst, continua fino al completamento senza essere interrotto, anche se nel frattempo arriva un processo con burst più breve. SRTF (SJF preemptive): se durante l'esecuzione di un processo arriva un nuovo processo il cui CPU burst è più breve del tempo rimanente del processo corrente, quest'ultimo viene interrotto (preemption) e il nuovo processo viene eseguito. SRTF ha tempi medi di attesa inferiori a STF ma maggiore overhead di context switch.

Nello scheduler SRTF il tempo residuo può diventare negativo. Perché?

> *(2022-07-20 c)*

Nello scheduler SRTF il tempo residuo può diventare negativo per una **sottostima del tempo di completamento** del processo. Si verifica quando l'algoritmo sottostima la quantità di tempo necessaria e lo scheduler passa ad un altro processo prima che il processo corrente sia effettivamente completato.

Come funziona la tecnica dell'aging? Come evita la starvation?

> *(2018-01-22 d)*

L'aging è una tecnica di scheduling a priorità dinamica che incrementa gradualmente la priorità dei processi che attendono nella coda ready da molto tempo. Un processo a bassa priorità, man mano che accumula tempo di attesa, vede la sua priorità salire progressivamente. A un certo punto la sua priorità diventa sufficientemente alta da superare quella di altri processi e lo scheduler lo seleziona per l'esecuzione. Questo garantisce che nessun processo venga posticipato indefinitamente, prevenendo la starvation che affliggerebbe uno scheduler a priorità statica.

Confrontare scheduler a priorità statica e round robin

> *(2015-01-20 c, 2015-02-14 3)*

Scheduler a priorità statica: ogni processo ha una priorità fissa assegnata alla creazione. Vantaggi: semplice da implementare, permette di favorire processi critici o di sistema. Svantaggi: i processi a bassa priorità possono soffrire starvation (non ottenere mai la CPU se ci sono sempre processi a priorità più alta pronti). Round Robin: tutti i processi hanno la stessa priorità e ricevono quanto di tempo uguale. Vantaggi: equità perfetta, nessuna starvation, buona reattività. Svantaggi: non permette di distinguere tra processi urgenti e meno urgenti, i processi I/O-bound possono essere penalizzati perché usano poco il quanto ma poi vanno in fondo alla coda.

Confrontare scheduler Round Robin e FIFO in un sistema batch monoprocessore

> *(2019-01-15 d)*

FIFO (FCFS): il primo processo arrivato esegue fino al completamento, poi il successivo. Vantaggi: overhead minimo (pochi context switch), throughput potenzialmente elevato, semplice. Svantaggi: effetto convoglio — un processo CPU-bound lungo blocca tutti i processi brevi dietro di sé, causando tempi di attesa medi molto alti. Round Robin: i processi si alternano a quanto di tempo. Vantaggi: evita l'effetto convoglio, tutti i processi fanno progressi. Svantaggi: maggiore overhead per context switch frequenti. In un sistema batch puro (senza interazione utente), l'overhead di Round Robin non è giustificato e FIFO è spesso la scelta migliore.

Quali sono gli interrupt hardware e trap più frequenti?

> *(2023-01-18 a)*

Interrupt hardware più frequenti: (1) interrupt del timer/clock — usato per lo scheduling preemptive, time slice e aggiornamento dell'orologio di sistema, generato a frequenza fissa (es. 100-1000 Hz); (2) interrupt del controller disco — segnala il completamento di operazioni I/O su memoria secondaria; (3) interrupt di rete — segnala l'arrivo di pacchetti sulla scheda di rete. Trap più frequenti: (1) page fault — accesso a pagina non presente in memoria principale; (2) system call — richiesta di servizi al kernel da parte di processi utente; (3) eccezioni aritmetiche — divisione per zero, overflow.

🔧 Varie (System Call, Kernel, Deadlock, POSIX)

Differenza fra una funzione di libreria (es. sprintf) e una system call (es. write)?

> *(2022-09-06 d)*

Una funzione di libreria come sprintf esegue interamente in user mode: manipola dati nello spazio di indirizzamento del processo senza mai passare per il kernel. Una system call come write richiede l'intervento del kernel per accedere a risorse protette (file, dispositivi I/O): causa una trap software che commuta il processore in kernel mode, il kernel esegue l'operazione richiesta, e poi ritorna in user mode. Le system call sono molto più costose in termini di tempo a causa del mode switch (salvataggio/ripristino contesto, potenziale context switch).

Come vengono passati i parametri a una system call e come ne viene ritornato il valore?

> *(2023-06-13 a)*

I parametri vengono messi in registri specifici della CPU. La chiamata della system call causa una trap, che viene catturata dal SO. Alla fine dell'esecuzione, il kernel pone il valore di ritorno in un altro registro.

Perché una system call non può essere implementata come una normale chiamata di funzione?

> *(2019-07-15 a)*

Perché una system call deve eseguire operazioni privilegiate (accesso a dispositivi I/O, modifica della MMU, manipolazione di strutture dati del kernel) che in user mode non sono consentite dall'hardware. Una normale chiamata di funzione non cambia il livello di privilegio del processore: rimane in user mode e qualsiasi tentativo di eseguire istruzioni privilegiate causerebbe una trap di protezione. Per passare in kernel mode serve esplicitamente una trap/interrupt software, che solo il meccanismo delle system call può innescare in modo controllato.

Esistono SO con un solo livello di privilegio (kernel mode = user mode)? Conseguenze?

> *(2018-09-19 d)*

Sì, esistono. La conseguenza è che non si possono implementare meccanismi di protezione. Un processo utente ha gli stessi privilegi del kernel, quindi può eseguire qualsiasi istruzione (incluso modificare il filesystem, accedere a memoria di altri processi, etc.).

Un sistema operativo a microkernel è più flessibile, più affidabile ma meno efficiente di un kernel monolitico. Perché?

> *(2019-05-18 c)*

Più flessibile: i servizi del SO (file system, driver, networking) girano come processi utente separati, facilmente sostituibili, aggiornabili o riconfigurabili senza ricompilare o riavviare il kernel. Più affidabile: un crash in un servizio (es. driver della scheda audio) non abbatte l'intero sistema — il processo servizio può essere riavviato isolatamente mentre il kernel continua a funzionare. Meno efficiente: la comunicazione tra servizi e kernel richiede message passing (IPC) attraverso il microkernel, con overhead di commutazione di contesto, copia dei messaggi tra spazi di indirizzamento e cambi di modalità, molto più costosi delle semplici chiamate a funzione dirette del kernel monolitico.

Perché il compilatore C è scritto in C? Come si compila il compilatore C senza compilatore C?

> *(2022-09-06 a)*

Il compilatore C è scritto in C per portabilità e manutenibilità: anziché riscrivere il compilatore in assembly per ogni nuova architettura, basta compilare il sorgente C del compilatore con un compilatore C già esistente sulla macchina target. Il bootstrap iniziale funziona così: la primissima versione del compilatore viene scritta in assembly (o in un altro linguaggio) per una macchina esistente. Una volta che questo compilatore rudimentale funziona, può compilare una versione più completa del compilatore scritta in C. Da quel momento in poi, il compilatore si auto-compila (self-hosting): ogni nuova versione del compilatore C viene compilata dalla versione precedente.

Come si fa a risolvere una situazione di deadlock?

> *(2022-06-21 b)*

Tre strategie principali. Prevenzione: eliminare strutturalmente una delle 4 condizioni necessarie per il deadlock (es. allocazione totale di tutte le risorse all'inizio, o allocazione gerarchica per rompere l'attesa circolare). Evitamento: a ogni richiesta di risorse, verificare se lo stato risultante è safe tramite l'algoritmo del banchiere; se unsafe, la richiesta viene ritardata. Rilevamento e recovery: periodicamente o su richiesta verificare la presenza di deadlock (es. riducendo il grafo di Holt) e intervenire terminando uno o più processi nel ciclo, oppure facendo rollback a checkpoint precedenti.

Come fa l'allocazione gerarchica ad evitare deadlock?

> *(2021-07-21 c)*

L'allocazione gerarchica **evita che si verifichi la condizione di attesa circolare**, una delle 4 condizioni necessarie per il deadlock. Se un processo P usa la risorsa r1 e attende r2 (usata da Q), l'attesa circolare richiederebbe che Q sia in attesa di r1. Ma con allocazione gerarchica, o r1 ha priorità minore di r2 (quindi Q dovrebbe rilasciare r2), o viceversa (P dovrebbe rilasciare r1). In entrambi i casi il ciclo si rompe.

Condizioni necessarie per il deadlock

> *(2006-06-19 6)*

Le quattro condizioni necessarie (devono valere tutte contemporaneamente) sono: (1) Mutua esclusione — le risorse non sono condivisibili, solo un processo alla volta può usarle; (2) Hold and Wait — un processo trattiene risorse già assegnate mentre attende di ottenerne altre; (3) Assenza di prerilascio (No Preemption) — le risorse non possono essere sottratte forzatamente al processo che le detiene; (4) Attesa circolare — esiste un ciclo di processi $P_1 \to P_2 \to \ldots \to P_n \to P_1$ in cui ogni P_i attende risorse trattenute da P_{i+1}.

Dimostrare che un ciclo in un grafo di Holt multirisorsa NON è condizione sufficiente per deadlock

> *(2018-07-17 d)*

Esempio: Siano P, Q, R processi. Sia A con due istanze a1, a2. Sia B con una istanza b1. Se P ha a1 e chiede B; Q ha b1 e chiede A; R ha a2. Abbiamo un **ciclo** nel grafo, ma **non un knot**: è completamente riducibile facendo terminare prima R, poi Q, poi P. Quindi **non c'è deadlock** nonostante il ciclo.

Cos'è un knot in un grafo? Quale teorema lega knot e deadlock?

> *(2015-02-14 4)*

Dato un nodo n in un grafo diretto, sia R(n) l'insieme dei nodi raggiungibili da n. Un knot è un sottoinsieme K di nodi tale che per ogni nodo n appartenente a K, R(n) = K: in altre parole, da qualsiasi nodo del knot si possono raggiungere tutti e soli i nodi del knot stesso. È un insieme "chiuso" da cui, una volta entrati, non si può più uscire. Teorema: in un grafo di Holt con al massimo una richiesta sospesa per processo, esiste deadlock se e solo se esiste un knot nel grafo.

Come funziona il meccanismo di DMA?

> *(2006-06-19 5)*

Il SO programma il controller DMA specificando: indirizzo di partenza in memoria centrale, indirizzo di destinazione sul dispositivo (o viceversa), numero di byte da trasferire e direzione del trasferimento (lettura/scrittura). Il controller DMA prende il controllo del bus di sistema (cycle stealing: ruba cicli di bus alla CPU senza bloccarla completamente), trasferisce i dati direttamente tra dispositivo e memoria centrale, e al termine del trasferimento genera un interrupt per segnalare il completamento. Durante l'intero trasferimento, la CPU è libera di eseguire altri processi.

Spiegare il meccanismo di calcolo approssimato della lunghezza del CPU burst in SJF

> *(2006-06-19 2)*

Poiché il vero CPU burst futuro non è noto, si usa una media esponenziale dei burst passati: $T_{n+1} = \alpha \cdot t_n + (1-\alpha) \cdot T_n$, dove t_n è la durata effettiva dell'n-esimo CPU burst appena concluso, T_n è la stima precedente, e α ($0 \leq \alpha \leq 1$) è un parametro che controlla il peso relativo tra storia recente e storia passata. Espandendo la formula: $T_{n+1} = \alpha \cdot t_n + \alpha(1-\alpha) \cdot t_{n-1} + \alpha(1-\alpha)^2 \cdot t_{n-2} + \ldots$, una serie geometrica dove i burst più vecchi hanno peso esponenzialmente decrescente. Con $\alpha=1$, conta solo l'ultimo burst; con $\alpha=0$, la stima non cambia mai.

Meccanismo dell'allocazione indicizzata in UNIX

> *(2006-06-19 4)*

UNIX usa un sistema misto basato su i-node (index node). Ogni i-node contiene gli attributi del file e un array di puntatori a blocchi dati. Tipicamente: 12-15 puntatori diretti (per file piccoli, accesso immediato), 1 puntatore indiretto singolo (punta a un blocco pieno di puntatori a blocchi dati), 1 puntatore indiretto doppio (punta a un blocco di puntatori a blocchi di puntatori), e 1 puntatore indiretto triplo. Questo schema multilivello permette di gestire efficientemente file di qualsiasi dimensione: i file piccoli (fino a ~48KB con blocchi 4KB) usano solo i puntatori diretti con un solo accesso a disco, mentre file enormi sfruttano i livelli di indirezione multipla.

Quali sono le differenze fra le operazioni P/V dei semafori e wait/signal delle variabili di condizione?

> *(2013-06-21 b)*

Le operazioni P (wait) e V (signal) sui semafori sono atomiche e operano su un contatore intero: P attende che il contatore sia >0 e lo decrementa, V incrementa il contatore (anche senza thread in attesa). Le variabili di condizione operano invece all'interno di un monitor: wait() rilascia atomicamente il lock del monitor e mette il thread in attesa sulla condizione; signal() risveglia un thread in attesa, ma il thread segnalante mantiene il lock del monitor. Differenza cruciale: un signal su variabile di condizione senza thread in attesa va perso (non ha memoria), mentre una V su semaforo incrementa il contatore e il prossimo thread che farà P lo troverà >0.

Perché il message passing asincrono e sincrono non hanno lo stesso potere espressivo?

> *(2018-09-19 c)*

Il message passing sincrono (rendezvous) è bloccante: sia sender che receiver si bloccano fino al completamento dello scambio. Quello asincrono non è bloccante: il sender prosegue immediatamente dopo aver depositato il messaggio. Con l'asincrono serve un buffer (coda messaggi) di dimensione potenzialmente illimitata. Per simulare il comportamento sincrono usando primitive asincrone, il sender dovrebbe inviare e poi attendere un acknowledgment dal receiver, il che richiede un secondo canale di comunicazione e introduce complessità aggiuntiva. Inoltre, l'operazione send() è intrinsecamente non trasformabile in modo semplice. Questa asimmetria fa sì che i due modelli non siano equivalenti in potere espressivo.

📊 Statistiche

Argomento	N. domande	Con risposta
Memoria / Paginazione / Trashing	28	28
File System / FAT / i-node / RAID	25	25
Sicurezza / ACL / Capability / Password	12	12
I/O / DMA / Interrupt / Spinlock	10	10
Scheduling	12	12
Varie / Deadlock / System Call	12	12
Totale	**~99**	**~99**

🔗 Fonti

- **Soluzioni repo**: `~/Documents/hermes/sistemi-operativi/repo/prove/scritto/*soluzione*`
- **PDF Notion Luizo**: `~/Documents/hermes/sistemi-operativi/teoria-notion/notion-luizo-sistemi-operativi.pdf` (teoria pp.1-113)
- **Raccolta raw**: `~/Documents/hermes/sistemi-operativi/repo/prove/scritto/G2_RACCOLTA_RISPOSTE.txt`
- **Piano di Studi SO**: Piano di Studi

🆕 Appendice: Esami 2023-2026 dal sito Davoli (14 esami)

> *Domande estratte direttamente dai PDF del sito ufficiale di Renzo Davoli.*
> *Questi esami NON erano presenti nel repo cartabinaria. Aggiunti il 25/05/2026.*
> *Risposte generate il 25/05/2026 usando la teoria da teoria_G2_SO.txt.*

2026.02.06

a) In una tabella FAT possono esistere elementi di uguale valore? (non considerando l'elemento nullo che contraddistingue la fine del file) Se sì: in quali casi? Se no: quali effetti avrebbe?

No, in condizioni normali ogni entry della FAT (escluso il marcatore EOF) deve avere un valore univoco, perché ogni entry rappresenta il puntatore al cluster successivo di un singolo file. Se due entry avessero lo stesso valore, si creerebbe un file "cross-linked": due file condividerebbero la stessa catena di cluster, e le modifiche a un file corromperebbero l'altro. Questo è uno stato di incoerenza del file system, tipicamente riparato da fsck/scandisk che duplica i cluster condivisi.

b) Trovare, se possibile, un grafo di Holt che abbia 5 nodi, tutte le risorse abbiano molteplicità due e tutti i processi siano in deadlock.

Sì, è possibile. Si usano 3 processi (P1, P2, P3) e 2 risorse (R1, R2), ciascuna con molteplicità 2. Assegnazioni: P1 detiene 1×R1 e richiede 1×R2; P2 detiene 2×R2 e richiede 1×R1; P3 detiene 1×R1 e richiede 1×R2. R1 è completamente allocata (P1=1, P3=1, libere=0) e R2 pure (P2=2, libere=0). Tutti e tre i processi hanno richieste insoddisfacibili → deadlock totale. Il grafo contiene un knot che include tutti i nodi.

c) Perché i metodi di sincronizzazione tipo test&set (spinlock) assumono grande rilevanza nella scrittura di sistemi operativi multiprocessore?

Nei sistemi SMP (multiprocessore), più CPU possono eseguire codice kernel simultaneamente e accedere alle stesse strutture dati condivise (code dei processi, tabelle pagine, buffer cache). In kernel mode non si può mettere un thread in sleep (non c'è un processo utente con PCB da sospendere), quindi l'attesa attiva con spinlock è l'unica

opzione per la mutua esclusione. Disabilitare gli interrupt bloccherebbe solo la CPU corrente, non le altre, quindi non sarebbe sufficiente. Lo spinlock, implementato con istruzioni atomiche test-and-set, protegge le sezioni critiche in modo efficiente per attese brevi.

d) Perché è difficile revocare una autorizzazione fornita tramite capability?

La capability è memorizzata dal processo stesso (è un "biglietto" che il kernel consegna al processo), non in una struttura dati centralizzata del sistema. Per revocarla, il kernel dovrebbe conoscere tutti i processi che ne possiedono una copia e rimuoverla da ciascuno, ma non esiste un elenco centralizzato di chi detiene quali capability. Con le ACL, invece, i diritti sono memorizzati nell'oggetto stesso (es. nell'i-node del file), quindi la revoca è immediata: basta modificare la lista associata all'oggetto.

2026.01.09

a) Quali problemi possono accadere se si utilizza un file system ext2 dove un file ha un numero di riferimenti errato?

Se il link count è maggiore del reale: il file system crede ci siano più riferimenti di quanti ne esistano realmente. Anche dopo la rimozione di tutti i link effettivi, il contatore non raggiunge zero e il file non viene mai deallocato → spazio su disco perso permanentemente. Se il link count è minore del reale: l'i-node e i blocchi dati possono essere deallocati prematuramente quando il contatore arriva a zero, mentre esistono ancora directory entry (hard link) validi che puntano a quell'i-node. Questi diventano dangling reference verso blocchi ormai liberi o riassegnati, causando potenziale corruzione dati.

b) Siano dati due array di dischi identici, il primo RAID1, il secondo RAID5. In quale array l'operazione di lettura è statisticamente più veloce?

RAID1 è statisticamente più veloce in lettura. In RAID1 ogni blocco è duplicato su due dischi, quindi tutte le testine possono servire richieste di lettura in parallelo: con N dischi si ha una capacità di lettura pari a N dischi. In RAID5 con N dischi, solo N−1 dischi contengono dati (uno è dedicato alla parità), quindi la capacità di lettura parallela è N−1. Inoltre RAID1 può schedulare ogni richiesta sul disco con il minor tempo di seek tra le due copie, riducendo ulteriormente la latenza media. RAID1 è quindi superiore in lettura, ma RAID5 offre più spazio utilizzabile a parità di dischi.

c) Descrizione errata dell'algoritmo del banchiere da correggere: "Quando un processo richiede risorse, l'algoritmo controlla se lo stato rimarrebbe safe: se sì assegna, se no sospende. Quando un processo restituisce risorse si controlla se il nuovo stato consente di soddisfare operazioni sospese." Perché è errata? Come correggerla?

L'errore principale è che l'algoritmo del banchiere viene invocato solo quando la richiesta non può essere soddisfatta immediatamente (risorse insufficienti), non per ogni richiesta. Se le risorse sono disponibili subito, si assegnano direttamente. Inoltre, in caso di stato unsafe, il processo non viene "sospeso" ma la richiesta viene semplicemente ritardata: il processo rimane in attesa finché altri processi non rilasciano risorse sufficienti. Correzione: "Quando un processo richiede risorse e queste non sono immediatamente disponibili, l'algoritmo simula l'assegnazione e verifica se lo stato risultante è safe. Se safe, assegna; se unsafe, non assegna e il processo attende. Quando un processo restituisce risorse, si rivalutano le richieste in attesa."

d) A cosa serve e come funziona il meccanismo di Aging negli scheduler a priorità dinamica?

L'aging serve a prevenire la starvation: un processo a bassa priorità che non viene mai eseguito perché sempre preceduto da processi a priorità più alta. Il meccanismo incrementa gradualmente la priorità del processo proporzionalmente al tempo trascorso in coda ready. Quando la priorità accumulata diventa sufficientemente alta da superare quella degli altri processi, lo scheduler seleziona finalmente il processo. Questo garantisce che ogni processo ottenga la CPU entro un tempo finito, senza richiedere all'utente di assegnare manualmente priorità elevate a tutti.

2025.09.05

a) Quando è sconsigliato usare un meccanismo di protezione di accesso di tipo Access Control List?

Le ACL sono sconsigliate quando l'elenco di utenti/gruppi con diritti su un oggetto è molto lungo, poiché la verifica richiede la scansione della lista a ogni accesso, con overhead crescente. Sono anche inadatte a sistemi distribuiti

dove l'oggetto non è localmente accessibile e la lista va replicata o consultata remotamente. Infine, quando serve determinare rapidamente "cosa può fare un utente" nel sistema, le ACL richiedono di scandire tutti gli oggetti, mentre le capability (visione per righe) rispondono immediatamente. In questi scenari, un sistema basato su capability o su role-based access control (RBAC) può essere preferibile.

b) Qual è la differenza fra una funzione di libreria e una system call? Perché sono necessarie le system call?

Una funzione di libreria (es. `sprintf`) esegue interamente in user mode manipolando dati nello spazio di indirizzamento del processo, senza mai passare per il kernel. Una system call (es. `write`) richiede l'intervento del kernel per accedere a risorse protette: causa una trap software che commuta il processore in kernel mode. Le system call sono necessarie perché i processi utente non possono eseguire istruzioni privilegiate (accesso a dispositivi I/O, modifica della MMU, manipolazione di strutture kernel). Solo il kernel, in modalità privilegiata, può operare sull'hardware in modo sicuro e controllato.

c) Il problema del deadlock può essere risolto con tecniche di checkpointing & rollback?

Sì, il checkpointing & rollback è una tecnica di deadlock recovery (non prevention o avoidance). Periodicamente il SO salva su disco lo stato dei processi (checkpoint). Quando viene rilevato un deadlock, si fa il rollback di uno o più processi coinvolti a un checkpoint precedente al deadlock, rilasciando le risorse che trattenevano. I processi ripartono da quel punto e il deadlock si risolve. Tuttavia questa tecnica ha dei costi: overhead di I/O per i checkpoint, potenziale perdita di lavoro (tutto ciò che è stato fatto dopo l'ultimo checkpoint), e complessità nel garantire che il rollback non causi effetti collaterali su processi non coinvolti.

d) Dati N dischi con le stesse caratteristiche, il tempo medio di accesso è minore con RAID0 o RAID5? Discutere prestazioni in lettura e scrittura.

In lettura, RAID0 e RAID5 hanno prestazioni simili: RAID0 usa tutti gli N dischi per dati, RAID5 usa N−1 dischi (uno per parità), quindi RAID0 ha un leggero vantaggio. In scrittura, RAID0 è nettamente superiore: scrive direttamente i dati senza overhead. RAID5 deve eseguire un ciclo read-modify-write: leggere i vecchi dati e la vecchia parità, calcolare la nuova parità (XOR), scrivere nuovi dati e nuova parità. Questo richiede 4 accessi a disco per ogni scrittura, rendendo RAID5 molto più lento in scrittura. Il tempo medio di accesso è quindi minore con RAID0 in entrambi i casi, ma RAID5 offre tolleranza a un guasto che RAID0 non ha.

2025.07.21

a) Perché durante la compattazione di memoria occorre sospendere l'esecuzione dei processi coinvolti?

La compattazione sposta fisicamente i processi in memoria per riunire i buchi liberi in un unico blocco contiguo. Se un processo continuasse a eseguire durante lo spostamento, i suoi puntatori e riferimenti a indirizzi di memoria diventerebbero inconsistenti (punterebbero alla vecchia posizione, ora libera o occupata da altri dati). Sospendere i processi garantisce che non accedano alla memoria mentre viene rilocata. Una volta completato lo spostamento, la MMU viene aggiornata con i nuovi indirizzi fisici e i processi possono riprendere. È necessaria inoltre la rilocabilità dinamica (binding a esecuzione) per poter spostare i processi.

b) Perché nell'i-node non è presente il nome del file?

Perché tramite hard link è possibile avere più nomi diversi che puntano allo stesso i-node. Se il nome fosse memorizzato nell'i-node, ci sarebbe spazio per un solo nome, rendendo impossibile avere più hard link. Il nome del file è invece memorizzato nelle directory: ogni directory entry associa un nome a un numero di i-node. Questa separazione permette a un i-node di avere un numero arbitrario di nomi (link count > 1), tutti equivalenti e indistinguibili, che condividono gli stessi blocchi dati.

c) Perché il microkernel è meno efficiente del kernel monolitico? Perché è più flessibile e sicuro?

Meno efficiente perché la comunicazione tra servizi (file system, driver, networking) e kernel richiede message passing (IPC) attraverso il microkernel, con overhead di commutazione di contesto, copia dei messaggi tra spazi di indirizzamento e cambi di modalità utente/kernel. In un monolitico, le stesse operazioni sono semplici chiamate a funzione dirette. Più flessibile perché i servizi girano come processi utente separati, facilmente sostituibili o riconfigurabili senza ricompilare il kernel. Più sicuro/affidabile perché un crash in un servizio (es. driver audio) non abbatte l'intero sistema: il microkernel lo isola e può riavviarlo.

d) Nella memorizzazione da parte dei processi utente delle capability per il controllo di accesso può essere usato un algoritmo di crittografia a singola chiave o occorre un algoritmo a doppia chiave? Perché?

Basta la crittografia a singola chiave (simmetrica). Il kernel è sia emittente che verificatore della capability: cifra la tupla `<oggetto, diritti>` con la propria chiave segreta e la consegna al processo. Quando il processo presenta la capability per un accesso, il kernel la decifra con la stessa chiave e ne verifica l'autenticità. La crittografia asimmetrica sarebbe superflua (non c'è scambio di chiavi tra entità diverse, il kernel comunica solo con sé stesso) e molto più costosa computazionalmente, introducendo latenza inappropriata a ogni controllo di accesso.

2025.06.23

a) L'algoritmo LOOK (dell'ascensore) vs C-LOOK: in quali casi si preferisce C-LOOK a LOOK?

C-LOOK si preferisce quando serve garantire equità ed evitare starvation. LOOK serve richieste in entrambe le direzioni: la testina inverte quando non ci sono più richieste nella direzione corrente, ma le richieste ai cilindri estremi possono attendere a lungo se continuano ad arrivare richieste nella direzione opposta. C-LOOK serve solo in una direzione (tipicamente cilindri crescenti), poi torna rapidamente all'inizio. Questo si implementa con due sottocode alternate — mentre si serve la prima coda, le nuove richieste vanno nella seconda, garantendo che ogni richiesta venga servita entro un tempo massimo.

b) Spiegare quali attacchi sono possibili se non viene usato il meccanismo del "sale" (salt) nella memorizzazione delle password criptate.

Senza salt, un attaccante può usare rainbow table (tabelle precalcolate che mappano hash → password in chiaro): calcolate una volta, funzionano contro qualsiasi sistema che usa lo stesso algoritmo di hash. Basta confrontare gli hash del file shadow con la tabella per trovare immediatamente le password deboli. Inoltre, due utenti con la stessa password avranno lo stesso hash, rivelando all'attaccante che condividono la password. Il salt (stringa casuale concatenata prima dell'hashing) rende ogni hash unico anche per password identiche e rende inutili le rainbow table precalcolate.

c) L'algoritmo del Banchiere evita deadlock ritardando richieste che porterebbero a stato unsafe. Basta questa regola a evitare anche starvation?

No, l'algoritmo del banchiere evita deadlock ma non garantisce l'assenza di starvation. Un processo potrebbe essere ripetutamente scavalcato: ogni volta che richiede risorse, un altro processo le ottiene prima, e quando finalmente sono disponibili, la verifica di safety fallisce perché altri processi hanno nel frattempo allocato risorse. Il processo rimane indefinitamente in attesa (starvation) anche se il sistema non è in deadlock. L'algoritmo del banchiere garantisce solo che, se esiste una sequenza safe, il sistema potrà soddisfare tutte le richieste in tempo finito — ma non specifica in quale ordine, quindi un processo può essere sempre "l'ultimo".

d) Perché è necessario usare spinlock in sistemi multiprocessore per implementare kernel SMP?

In un sistema SMP, più CPU eseguono codice kernel simultaneamente e possono accedere alle stesse strutture dati condivise. Senza protezione, si verificherebbero race condition. In kernel mode non esiste un processo utente con PCB da sospendere, quindi non si possono usare meccanismi di attesa passiva (sleep/wakeup). Lo spinlock, con attesa attiva tramite istruzioni atomiche test-and-set, è l'unica soluzione. Disabilitare gli interrupt su una CPU non impedisce alle altre CPU di accedere alle stesse strutture. Lo spinlock è efficiente per sezioni critiche brevi in kernel mode.

2025.05.28

a) Perché è difficile revocare un diritto di accesso se espresso come capability rispetto a una ACL?

La capability risiede nello spazio di memoria del processo, non in una struttura centralizzata del SO. Per revocarla, il kernel dovrebbe individuare e invalidare tutte le copie distribuite tra i processi, ma non mantiene un registro di chi possiede quali capability. Nelle ACL, i diritti sono memorizzati nell'oggetto protetto (es. i-node del file): revocare significa semplicemente rimuovere una entry dalla lista, operazione immediata e centralizzata. Questa è la differenza fondamentale tra visione per righe (capability, difficile revoca) e visione per colonne (ACL, facile revoca) della matrice di accesso.

b) Si può implementare la memoria virtuale con paginazione a richiesta in un sistema con MMU priva di TLB?

Sì, è possibile ma estremamente inefficiente. Senza TLB, ogni accesso in memoria richiede due accessi fisici: uno alla tabella delle pagine (per tradurre l'indirizzo logico in fisico) e uno al dato effettivo. Questo raddoppia il tempo di ogni accesso. Il TLB è una cache che evita il primo accesso nella maggior parte dei casi. I sistemi reali includono sempre un TLB perché senza di esso le prestazioni degraderebbero in modo inaccettabile, ma concettualmente la paginazione a richiesta funziona anche con la sola tabella pagine in memoria.

c) Perché non si può implementare la funzionalità di link fisico in un file system di tipo FAT?

FAT non ha una struttura equivalente all'i-node con contatore di riferimenti. In FAT, le informazioni sul file (nome, attributi, primo cluster) sono memorizzate nella directory entry, e la catena dei cluster è nella tabella FAT. Per creare un hard link servirebbe una seconda directory entry che punti allo stesso primo cluster, ma FAT non ha un meccanismo per contare quanti riferimenti puntano a una catena di cluster. Senza link count, cancellare un hard link libererebbe i cluster, rompendo l'altro link. Inoltre, FAT non distingue tra "nome del file" e "metadati del file" — tutto è nella directory entry, rendendo impossibile avere più nomi per lo stesso insieme di dati.

d) L'algoritmo del Banchiere evita deadlock. Sembra desiderabile, eppure viene raramente usato nei sistemi reali. Perché?

Il problema principale è che i processi devono dichiarare a priori il massimo numero di risorse di ciascuna classe di cui avranno bisogno, e questa informazione semplicemente non è nota nella maggior parte delle applicazioni reali. Si può stimare, ma mai con l'esattezza richiesta dall'algoritmo. Inoltre, in sistemi con molte classi di risorse, la verifica di safety ha complessità $O(n^2 \times m)$, costosa da eseguire frequentemente. Infine, molti SO moderni preferiscono tecniche di deadlock prevention più semplici (es. allocazione gerarchica) o, pragmaticamente, l'approccio dello struzzo: i deadlock sono rari e un reboot è più economico dell'overhead continuo della verifica.

2024.09.10

a) RAID61 (RAID1 di RAID6): qual è il livello di fault tolerance? E RAID16 (RAID6 di RAID1)?

RAID61 è uno stripe di RAID6 protetto da mirroring. RAID6 tollera fino a 2 guasti per stripe; il mirroring aggiunge un altro livello. In RAID61: ogni stripe può perdere 2 dischi, e il mirror consente di perdere un intero stripe (se l'altro sopravvive). Quindi tollera fino a 2 guasti sullo stesso stripe + il guasto completo dell'altro stripe, ma se si rompono 3+ dischi sullo stesso stripe il mirror non basta. RAID16 è RAID6 su un array di RAID1: tollera il guasto di 2 interi mirror (ogni mirror può perdere 1 disco) più guasti aggiuntivi su mirror diversi. RAID16 è più robusto: tollera il guasto di qualsiasi coppia di dischi e anche di molte combinazioni di 3+ guasti, purché non cadano più di 2 mirror completi.

b) L'amministratore crea una partizione su disco rotazionale con FAT da montare come /tmp. Buona o cattiva idea? Perché?

Cattiva idea. /tmp contiene file temporanei con frequenti creazioni, modifiche e cancellazioni. FAT, non avendo journaling, è vulnerabile a incoerenze in caso di crash durante operazioni su /tmp — servirebbe un lungo fsck/scandisk a ogni riavvio. Inoltre FAT causa frammentazione esterna significativa con file che crescono e decrescono frequentemente, degradando le performance su disco rotazionale dove la testina deve seguire catene di cluster sparse. Infine, FAT non supporta i permessi UNIX necessari per /tmp (sticky bit, permessi utente). Un file system con journaling (ext4, XFS) o un tmpfs in RAM sono scelte nettamente superiori.

c) Quali sono i benefici degli interrupt nidificati e quali le difficoltà per sviluppare un SO che li supporti?

Benefici: permettono di gestire interrupt ad alta priorità anche durante l'esecuzione di un handler per un interrupt a priorità inferiore, riducendo la latenza per dispositivi time-critical. Un controller disco veloce non deve attendere il completamento della gestione di un interrupt della tastiera. Difficoltà: richiedono stack multipli (uno per ogni livello di priorità) per evitare corruzione dello stato; le sezioni critiche negli handler devono essere protette con mascheramento selettivo degli interrupt; il debugging è molto più complesso a causa dell'esecuzione potenzialmente annidata e delle race condition; la progettazione deve garantire che handler a priorità più alta non causino starvation di quelli a priorità inferiore.

d) Perché aumentando la dimensione dell'area di memoria secondaria usata dalla memoria virtuale si corre rischio di trashing?

Aumentare lo spazio di swap consente di ospitare più processi in memoria virtuale, incrementando il grado di multiprogrammazione. Se il numero totale di frame fisici rimane invariato, la somma dei working set dei processi attivi può superare i frame disponibili — ogni processo genera continui page fault perché le sue pagine vengono rimpiazzate da altri processi prima che possa riutilizzarle. Il sistema entra in trashing: spende più tempo a gestire page fault (I/O su disco) che a eseguire codice utile. Paradossalmente, lo scheduler rileva CPU bassa e può caricare ulteriori processi, peggiorando ulteriormente la situazione.

2024.07.22

a) La ricostruzione della coerenza mediante journaling risolve solo alcuni tipi di incoerenze. Quali NON vengono gestite?

Il journaling NON gestisce: (1) le transazioni in coda ma non ancora confermate al momento del guasto — queste vengono perse definitivamente; (2) i guasti meccanici del disco (settori danneggiati, testine guaste) — il journal registra operazioni logiche, non protegge i dati fisicamente; (3) i danni causati da bug del file system stesso che scrivono dati errati ma semanticamente validi nel journal; (4) la corruzione silenziosa dei dati (bit rot) dove i dati su disco si degradano nel tempo senza che il sistema se ne accorga — per questo servono checksum come in ZFS/Btrfs.

b) A cosa serve partizionare un disco? Fornire esempi pratici.

Partizionare un disco serve a suddividere logicamente lo spazio per organizzazione, sicurezza e performance. Esempi pratici: (1) separare SO e dati utente (partizione `/` e `/home`): reinstallare il SO non cancella i documenti; (2) partizione `/tmp` con flag `noexec` e `nosuid` per sicurezza, impedendo esecuzione di codice malevolo; (3) partizione separata per `/var` (log, database, code di stampa): se i log riempiono il disco, il sistema su `/` continua a funzionare; (4) partizione di swap dedicata per memoria virtuale; (5) multi-boot: installare più SO sullo stesso disco, ciascuno nella propria partizione.

c) Il tempo rimanente calcolato dall'algoritmo SRTF può diventare negativo? (sì/no, perché?)

Sì, può diventare negativo. SRTF (Shortest Remaining Time First) stima il tempo rimanente usando una media esponenziale dei CPU burst precedenti. Se la stima è inferiore al tempo effettivamente necessario (sottostima), il processo esaurisce il tempo stimato ma non ha ancora completato il CPU burst. A quel punto il tempo rimanente calcolato diventa negativo, e lo scheduler può erroneamente passare a un altro processo, causando un context switch aggiuntivo e una penalizzazione ingiusta del processo in esecuzione. È un artefatto della natura predittiva (e imperfetta) della stima del CPU burst.

d) In quali casi è necessario compilare un kernel Linux specifico?

È necessario quando: (1) il sistema target ha un'architettura hardware non standard o embedded (es. ARM personalizzato, MIPS, RISC-V) che richiede driver e configurazioni specifiche non presenti nel kernel generico; (2) si sviluppano o testano nuovi driver, moduli kernel o modifiche al kernel stesso; (3) si deve creare un sistema minimale (es. dispositivi IoT, router) con solo i moduli necessari per ridurre l'occupazione di memoria e la superficie di attacco; (4) si porta Linux su un processore con nuovo instruction set, richiedendo la scrittura del codice assembly specifico per context switch, interrupt handling e MMU nel sottosistema `arch/`.

2024.05.29

a) Un controller DMA per i dischi migliora le prestazioni di un servizio di memoria virtuale con paginazione a richiesta?

Sì, in modo significativo. Durante un page fault, il paginatore deve caricare una pagina dal disco alla memoria: senza DMA, la CPU dovrebbe eseguire programmed I/O copiando ogni byte dal controller del disco alla RAM, restando occupata per tutto il trasferimento. Con DMA, il paginatore programma il controller specificando indirizzo del frame e dimensione, poi la CPU è libera di eseguire altri processi mentre il DMA trasferisce autonomamente l'intera pagina. Al termine, un interrupt segnala il completamento. Poiché i page fault sono operazioni frequenti e i trasferimenti sono di intere pagine (4KB+), il DMA riduce drasticamente il tempo di CPU sprecato in I/O.

b) Il meccanismo del sale (salt) serve per evitare attacchi a dizionario. Perché un sale di un solo byte può non essere sufficiente?

Un sale di 1 byte ha solo 256 valori possibili. Un attaccante può precalcolare 256 rainbow table (una per ogni possibile sale) e usarle contro qualsiasi password hashata con sale a 1 byte. L'attacco è solo 256 volte più costoso di un attacco senza sale — un overhead trascurabile con hardware moderno. Per essere efficace, il sale deve avere uno spazio sufficientemente grande (tipicamente 64-128 bit) in modo che il numero di rainbow table necessarie sia computazionalmente impraticabile (2^64 o più). Un sale di 1 byte vanifica quindi solo marginalmente il vantaggio delle rainbow table.

c) Un progettista dimezza il time slice di round robin a ogni processo che lo consuma completamente senza fare I/O. Quali problemi potrebbero emergere?

Il time slice diventerebbe rapidamente troppo breve per i processi CPU-bound, causando un numero eccessivo di context switch — la CPU passerebbe più tempo a salvare/ripristinare lo stato dei processi che a eseguire codice utile. Inoltre, i processi CPU-bound legittimi (es. calcoli scientifici, rendering) verrebbero ingiustamente penalizzati con time slice minuscoli, mentre processi I/O-bound manterrebbero time slice lunghi. Si creerebbe un circolo vizioso: processi CPU-bound generano più context switch, consumando CPU che potrebbe essere usata per esecuzione utile. Il sistema degraderebbe in modo significativo.

d) Differenza fra deadlock prevention e deadlock avoidance? L'allocazione gerarchica è prevention o avoidance? Perché?

La deadlock prevention elimina strutturalmente una delle quattro condizioni necessarie per il deadlock (mutua esclusione, hold-and-wait, no preemption, attesa circolare), rendendo il deadlock impossibile a prescindere dallo stato del sistema. La deadlock avoidance permette che tutte le condizioni possano verificarsi, ma controlla ogni richiesta di risorse per garantire che lo stato rimanga safe (es. algoritmo del banchiere). L'allocazione gerarchica è prevention perché rompe strutturalmente la condizione di attesa circolare: imponendo un ordinamento totale tra classi di risorse e permettendo ai processi di allocare solo risorse a priorità maggiore di quelle già possedute, rende impossibile la formazione di cicli nel grafo di attesa.

2024.02.13

a) Perché è necessario l'interval timer per implementare lo scheduler round robin?

L'interval timer è il meccanismo hardware che genera interrupt a intervalli regolari (es. ogni 10ms), permettendo al kernel di riprendere il controllo della CPU. In Round Robin, ogni processo ha un quanto di tempo (time slice). Allo scadere del quanto, l'interrupt del timer attiva lo scheduler, che salva lo stato del processo corrente e lo sposta in fondo alla coda ready, quindi carica il prossimo processo. Senza interval timer, Round Robin degenererebbe in FCFS/FIFO: un processo potrebbe monopolizzare la CPU indefinitamente perché il kernel non avrebbe modo di interromperlo forzatamente. Il timer è quindi il fondamento hardware della preemption.

b) Per accesso diretto (seek) a file grandi: allocazione concatenata o indicizzata è più efficiente? Perché?

L'allocazione indicizzata è molto più efficiente. Con allocazione concatenata, ogni blocco contiene un puntatore al successivo, quindi per raggiungere il blocco N-esimo occorre leggere tutti i blocchi precedenti sequenzialmente (accesso O(N)). Con allocazione indicizzata, il blocco indice contiene tutti i puntatori ai blocchi dati; per una seek, si consulta l'indice (caricato in memoria) e si ottiene immediatamente l'indirizzo del blocco desiderato. Il numero di accessi a disco per una seek è O(1) con l'indicizzata, O(N) con la concatenata. Per file grandi la differenza diventa enorme.

c) Cosa fa il SO se una richiesta all'algoritmo del banchiere porta a uno stato unsafe? Perché?

Se la richiesta porterebbe a uno stato unsafe, il SO **non assegna le risorse** e il processo rimane in attesa. Il motivo: uno stato unsafe non è ancora deadlock, ma potrebbe evolvere in deadlock se i processi fanno richieste sfavorevoli. L'algoritmo del banchiere è conservativo: rifiuta qualsiasi transizione che renda il sistema unsafe, anche se il deadlock non è certo. Questo garantisce che, fintanto che i processi rispettano i bound dichiarati e restituiscono le risorse, il sistema non entrerà mai in deadlock. Le risorse vengono concesse solo quando la verifica produce uno stato safe.

d) Confrontare RAID5 con RAID50: capacità utilizzabile, dischi guasti tollerati, vantaggi/svantaggi.

RAID5 con N dischi: capacità N−1 dischi, tollera 1 guasto. RAID50 è uno stripe (RAID0) di due o più array RAID5. Con due gruppi RAID5 da M dischi ciascuno (totale 2M): capacità 2(M−1) dischi, tollera 1 guasto per gruppo (fino a

2 totali, ma solo se in gruppi diversi). Vantaggio RAID50: maggiori performance in lettura/scrittura grazie allo striping tra i gruppi RAID5, e maggiore fault tolerance se i guasti sono distribuiti. Svantaggio: richiede più dischi minimi (almeno 6: due gruppi da 3), capacità leggermente inferiore rispetto a RAID5 con lo stesso numero di dischi se i gruppi sono piccoli.

2024.01.17

a) Quali algoritmi di scheduling sono più adatti a sistemi interattivi: preemptive o non-preemptive? Perché?

Gli algoritmi preemptive sono nettamente più adatti ai sistemi interattivi. In un sistema interattivo, l'utente si aspetta risposte rapide alle proprie azioni (click, tasti premuti). Uno scheduler non-preemptive permetterebbe a un processo CPU-bound di monopolizzare il processore per tutto il suo CPU burst, bloccando l'interfaccia utente per secondi — inaccettabile. Gli algoritmi preemptive (Round Robin, scheduling a priorità con preemption) possono interrompere il processo in esecuzione allo scadere del time slice o all'arrivo di un processo a priorità più alta, garantendo bassa latenza di risposta e una percezione di reattività da parte dell'utente.

b) L'uso di SSD per memoria virtuale consentirebbe minori problemi di trashing? Perché è comunque sconsigliato?

L'SSD ridurrebbe la latenza dei page fault (tempi di accesso molto inferiori all'HDD), quindi il trashing sarebbe meno catastrofico in termini di throughput — i page fault verrebbero serviti più velocemente. Tuttavia è sconsigliato perché: (1) gli SSD hanno un numero limitato di cicli di scrittura, e il trashing (o anche il normale swap) genera moltissime scritture che consumano rapidamente la vita del disco; (2) la memoria virtuale con swap su SSD accelera l'usura senza offrire un vero rimedio al trashing — il problema di fondo (working set > frame disponibili) va risolto riducendo il grado di multiprogrammazione, non rendendo più veloce lo swap.

c) Mostrare almeno tre tipi di incongruenze rilevate durante il controllo di coerenza di un file system ext3.

(1) Blocco marcato come libero nella mappa di bit ma anche come occupato in un i-node (o viceversa: blocco in uso ma non riferito da alcun i-node) — incongruenza tra bitmap e i-node. (2) Link count errato: il contatore di riferimenti nell'i-node non corrisponde al numero effettivo di directory entry che puntano a quell'i-node (rilevato contando le occorrenze durante la scansione delle directory). (3) Un i-node con link count > 0 ma senza alcuna directory entry corrispondente (file "orfano" in lost+found) — oppure directory entry che puntano a i-node inesistenti o deallocati (dangling reference). fsck corregge queste incongruenze riparando bitmap, aggiornando link count, o spostando file orfani in lost+found.

d) Come funziona RAID60? Numero minimo di dischi, capacità utilizzabile, accessi per scrittura di un blocco.

RAID60 è uno stripe (RAID0) di più array RAID6. RAID6 richiede almeno 4 dischi (2 dati + 2 parità); con due gruppi RAID6 in striping servono almeno 8 dischi. Capacità utilizzabile: per G gruppi da M dischi ciascuno, capacità = G × (M−2) dischi. Con 8 dischi (2 gruppi da 4): capacità = 2 × (4−2) = 4 dischi. Accessi per scrittura: RAID6 richiede 6 accessi per scrittura (read-modify-write su dati e doppia parità), ma RAID60 non aggiunge overhead ulteriore — ogni scrittura interessa un solo gruppo RAID6. Quindi 6 accessi a disco per scrittura. RAID60 tollera 2 guasti per gruppo (fino a 4 totali se in gruppi diversi).

2023.09.11

a) Perché è necessario l'algoritmo della media esponenziale nello scheduler SJF? A cosa serve il parametro alpha?

L'algoritmo SJF (Shortest Job First) richiede di conoscere la lunghezza del prossimo CPU burst, che non è nota a priori. La media esponenziale la stima basandosi sui burst passati: $T_{n+1} = \alpha \cdot t_n + (1-\alpha) \cdot T_n$, dove t_n è l'ultimo burst effettivo e T_n è la stima precedente. Il parametro α ($0 \leq \alpha \leq 1$) controlla il peso tra storia recente e passata: $\alpha=1$ considera solo l'ultimo burst (alta reattività ai cambiamenti), $\alpha=0$ mantiene la stima iniziale invariata (ignora la storia). Valori tipici sono $\alpha=1/2$, che dà peso esponenzialmente decrescente ai burst più vecchi, bilanciando reattività e stabilità della stima.

b) Confrontare RAID10 e RAID6 con 4 dischi: capacità, fault tolerance, prestazioni.

Con 4 dischi: RAID10 (RAID1+0) ha 2 gruppi mirror in striping, capacità 2 dischi. RAID6 ha 2 dischi dati + 2 parità, capacità anch'essa 2 dischi. Fault tolerance: RAID10 tollera 1 guasto per mirror (fino a 2 totali, ma solo se su mirror diversi — se cadono entrambi i dischi dello stesso mirror, dati persi). RAID6 tollera qualsiasi coppia di dischi guasti, quindi è più robusto. Prestazioni: RAID10 è superiore in scrittura (nessun calcolo di parità, solo duplicazione), RAID6

ha overhead significativo (6 accessi per scrittura). In lettura, entrambi distribuiscono il carico su più dischi, con RAID10 leggermente favorito perché tutti i dischi contengono dati utili.

c) Perché i metodi di sincronizzazione tipo test&set (spinlock) assumono grande rilevanza nei sistemi multiprocessore?

Nei sistemi multiprocessore SMP, più CPU eseguono codice kernel simultaneamente e possono accedere alle stesse strutture dati condivise. In kernel mode non è possibile mettere un thread in sleep (manca un PCB di processo utente), quindi l'attesa attiva con spinlock è l'unica opzione praticabile per la mutua esclusione. Disabilitare gli interrupt protegge solo la CPU locale, non le altre. Lo spinlock, implementato con istruzioni atomiche test-and-set, è efficiente per sezioni critiche brevi perché la CPU in attesa non cede il controllo — a differenza del context switch, che in kernel mode sarebbe molto costoso o impossibile.

d) Perché ext2 è più efficiente nell'accesso diretto a file piccoli rispetto a file grandi?

Perché ext2 usa un sistema misto di puntatori nell'i-node: i primi 12 puntatori sono diretti e coprono i primi 12 blocchi del file (tipicamente 48KB con blocchi da 4KB). Per file che rientrano in questi blocchi, l'accesso a qualsiasi posizione richiede un solo accesso a disco (lettura dell'i-node + lettura del blocco dati tramite puntatore diretto). Per file grandi, occorre attraversare i livelli di indirezione: indiretto singolo (+1 accesso), doppio (+2 accessi), triplo (+3 accessi). I file piccoli beneficiano quindi di un accesso O(1) senza overhead di indirezione, mentre file grandi richiedono accessi aggiuntivi ai blocchi indice intermedi.

2023.07.19

a) Quando un sistema è in trashing il carico della CPU è alto o basso? Perché?

Il carico della CPU è basso. In trashing, il sistema spende più tempo nella paginazione (I/O su disco per gestire i page fault) che nell'esecuzione effettiva dei processi. I processi sono quasi sempre bloccati in attesa che i page fault vengano serviti, mentre la CPU resta inattiva. Paradossalmente, lo scheduler, rilevando CPU bassa, può decidere di caricare ulteriori processi pensando ci sia capacità residua, aumentando la contesa per i frame e peggiorando ulteriormente il trashing. Questa è la "trappola del trashing".

b) Quali sono i passi per portare Linux a una nuova architettura (nuovo processore con nuovo instruction set)?

(1) Scrivere il codice assembly specifico per l'architettura nella directory `arch/`: gestione del context switch (salvataggio/ripristino registri), routine di interrupt e trap, gestione della MMU (traduzione indirizzi, TLB), codice di bootstrap. (2) Implementare il supporto per le convenzioni di chiamata (calling convention) del nuovo processore. (3) Adattare o riscrivere i driver per i dispositivi specifici della piattaforma. (4) Scrivere il linker script e le definizioni per il layout di memoria. (5) Compilare il kernel con un cross-compiler che targettizzi il nuovo instruction set. (6) Testare su emulatore (QEMU) e poi su hardware reale.

c) Trovare, se possibile, un grafo di Holt con 6 nodi, risorse con molteplicità 2, tutti i processi in deadlock.

Sì, è possibile. Si usano 3 processi (P1, P2, P3) e 3 risorse (R1, R2, R3), ciascuna con molteplicità 2. Totale: 6 nodi. Assegnazioni: P1 detiene 1×R1 e 1×R2, richiede 1×R3; P2 detiene 2×R3, richiede 1×R1; P3 detiene 1×R1 e 1×R2, richiede 1×R3. R1 è allocata completamente (P1=1, P3=1, libere=0), R2 pure (P1=1, P3=1, libere=0), R3 pure (P2=2, libere=0). P1 e P3 attendono R3, P2 attende R1 — nessuno può progredire. C'è un knot che include tutti i nodi, confermando il deadlock. In alternativa, si può usare 4 processi e 2 risorse (4+2=6 nodi).

d) A cosa serve il meccanismo challenge/response nella gestione delle password? Come funziona?

Il challenge/response serve a evitare di trasmettere la password in chiaro (o il suo hash) sulla rete, prevenendo attacchi di tipo replay e sniffing. Funziona così: il server invia un challenge (numero casuale, nonce) al client; il client calcola una risposta combinando la password (nota solo al client e al server) con il challenge, tipicamente tramite una funzione hash: response = H(challenge, password); il server esegue lo stesso calcolo e confronta. Un attaccante che intercetta il challenge e la response non può riutilizzarli perché il prossimo challenge sarà diverso, e dal challenge+response non può risalire alla password grazie alla one-way function.

2023.06.01

a) Nei file system UNIX il nome del file non è nell'i-node. Dove è memorizzato? Quale operazione non sarebbe possibile se fosse nell'i-node?

Il nome del file è memorizzato nelle directory: ogni directory entry è una coppia `<nome_file, numero_i-node>`. Se il nome fosse nell'i-node, non sarebbe possibile creare hard link: un hard link crea una seconda directory entry con un nome diverso che punta allo stesso i-node. Se l'i-node contenesse il nome, ci sarebbe spazio per un solo nome e non si potrebbero avere più nomi per lo stesso file. La separazione tra nome (nella directory) e metadati/dati (nell'i-node) è la chiave che rende possibili gli hard link.

b) Da quale tipo di attacco protegge il meccanismo del sale (salt) nella memorizzazione delle password?

Protegge dagli attacchi a dizionario basati su rainbow table. Le rainbow table sono tabelle precalcolate che mappano hash → password in chiaro per tutti i possibili input di una funzione hash. Senza salt, un attaccante può usare la stessa rainbow table contro tutte le password del sistema. Il salt, essendo una stringa casuale diversa per ogni password, rende ogni hash unico anche a parità di password — l'attaccante dovrebbe generare una rainbow table separata per ogni possibile sale, rendendo l'attacco computazionalmente impraticabile se il sale è sufficientemente lungo. Protegge anche dal rilevamento di password identiche tra utenti diversi.

c) Proposta: time slice più corti per processi interattivi e più lunghi per processi batch in round robin. È una buona idea?

Non è una buona idea nel contesto di round robin puro, che per definizione assegna lo stesso quanto a tutti. La proposta descrive in realtà uno scheduling a priorità con code multiple (code con time slice diversi). In un sistema reale, questa è esattamente la strategia adottata: i processi interattivi (I/O-bound) ricevono priorità più alta e time slice brevi perché tipicamente si bloccano prima di esaurire il quanto, mentre i processi batch (CPU-bound) ricevono priorità più bassa e time slice più lunghi per ridurre l'overhead di context switch. Ma questo si implementa con scheduling multilivello, non con round robin puro.

d) Per applicare LRU occorrerebbe un hardware molto complesso e costoso. Perché?

LRU richiede di sapere, a ogni accesso in memoria, quale pagina è stata usata meno recentemente tra tutte quelle in memoria. Le implementazioni naive sono proibitive: (1) contatori per ogni pagina, incrementati a ogni accesso — servirebbero contatori molto grandi (overflow) e a ogni page fault occorrerebbe scandire tutte le pagine per trovare il contatore minimo; (2) stack hardware di pagine, dove ogni accesso sposta la pagina in cima — servirebbero 6 puntatori da aggiornare per ogni accesso in memoria. Entrambe le soluzioni richiederebbero hardware dedicato costoso e consumerebbero tempo a ogni singolo accesso, annullando i benefici della cache TLB. Per questo i SO reali usano approssimazioni di LRU (second chance, reference bit).

Giugno 2026

a) In quali casi il sistema operativo esegue l'algoritmo del banchiere? Cosa succede se il risultato è non-safe?

Il sistema esegue il test di safety prima di rendere definitiva un'operazione che potrebbe ridurre le risorse disponibili o aggiungere nuovi crediti massimi: ammissione di un nuovo processo (con credito massimo ed eventuale assegnazione iniziale) e richieste di risorse attualmente disponibili. Per una richiesta si controlla prima che non superi il credito residuo; se le risorse non sono disponibili la richiesta resta sospesa; altrimenti si fa un'assegnazione simulata e si esegue il test. Se safe l'assegnazione viene confermata, se non-safe viene annullata e la richiesta resta sospesa. Uno stato non-safe non è già in deadlock, ma non garantisce più che tutti i processi possano terminare. A ogni rilascio o terminazione le richieste sospese vengono riesaminate.

b) Nuovo processore csirv con nuovo instruction set: passi per portare Linux?

Linux è in gran parte in C, ma toolchain, kernel e programmi devono conoscere il nuovo ISA: (1) backend csirv in GCC/LLVM più assembler, linker e formato eseguibili: si ottiene un cross-compilatore su macchina già supportata; (2) ABI: registri, calling convention, stack, tipi, formato binari, numerazione system call; (3) port architetturale in arch/csirv: bootstrap, trap/interrupt, system call, context switch, atomiche/spinlock, timer, MMU/page table/TLB, SMP; (4) bootloader/firmware e driver essenziali (interrupt, timer, console, storage); (5) libc e user space cross-compilati con root filesystem; (6) avvio prima su emulatore. Il bootstrap non richiede un compilatore nativo: tutto nasce dal cross-compilatore.

c) In SRTF il tempo residuo può diventare negativo: perché? Come si comporta lo scheduler?

SRTF non conosce la durata reale del burst: usa una stima (media esponenziale dei burst precedenti) e calcola residuo = stima − tempo eseguito. Se il burst reale supera la stima, il residuo diventa negativo. Poiché SRTF sceglie il minimo, il valore negativo batte ogni altra stima: il processo continua fino al termine effettivo del burst (o l'implementazione satura a zero, stesso effetto). Alla fine la durata reale aggiorna la stima futura.

d) Quale attacco rende arduo il salt? Perché è difficile da individuare?

Gli attacchi offline a dizionario/forza bruta con hash precalcolati (rainbow table). Ogni password ha salt casuale diverso e si memorizzano salt e hash: anche password uguali danno hash diversi, l'attaccante deve ricalcolare tutto per ogni salt. È difficile da individuare perché, dopo il furto degli hash (es. /etc/shadow), tutto avviene offline sulla macchina dell'attaccante: nessun login, traffico o evento osservabile. Il salt non rende segreto l'hash: servono /etc/shadow protetto e funzioni di derivazione lente.

Luglio 2026

a) In trashing il carico della CPU è alto o basso, perché?

Basso: i processi non tengono il working set in RAM, generano continui page fault e restano bloccati in attesa del disco mentre la CPU resta senza ready. Paradosso: vedendo CPU bassa il sistema potrebbe caricare altri processi e peggiorare tutto; la contromisura è ridurre il grado di multiprogrammazione sospendendo processi.

b) Mostrare un grafo di Holt connesso con 2 knot da 5 nodi, max 1 richiesta per processo.

Costruzione da 13 nodi. Knot A {P1,P2,P3,R1,R2}: R1 (molteplicità 1) assegnata a P3; R2 (molteplicità 2) assegnata a P1 e P2; P1 e P2 richiedono R1, P3 richiede R2. Knot B {Q1,Q2,Q3,S1,S2} simmetrico. Collegamento: risorsa Rx con molteplicità 2 assegnata a Pa e Pb esterni, con Pa che richiede R1 e Pb che richiede S1. Gli archi entrano nei knot ma nessuno esce: restano knot distinti, grafo connesso, ogni processo ha esattamente una richiesta. Con max 1 richiesta per processo, il knot equivale al deadlock.

c) Perché in Unix esistono fork ed exec separate? Benefici?

fork crea un processo duplicando il chiamante (stesso punto, PID diversi); exec sostituisce programma e spazio di indirizzamento mantenendo il PID. La separazione serve alla shell: fork, poi nel figlio configura redirezioni/pipe/directory/limiti, poi exec del comando, mentre il padre attende o lancia pipeline. I meccanismi sono ortogonali (figlio senza exec, exec senza fork) e con copy-on-write la fork prima di exec non copia quasi niente.

d) Come trova fsck i file inaccessibili tramite pathname (es. ext2)?

Non usa i pathname: attraversa ricorsivamente le directory dalla radice marcando gli i-node raggiungibili, poi li confronta con gli i-node marcati allocati. Un i-node allocato ma irraggiungibile è orfano: se coerente riceve una entry in lost+found, altrimenti viene deallocato. Nella stessa scansione ricalcola i link count e confronta i blocchi usati con le bitmap.